C# 学习

前言：这份笔记只围绕 **C#/.NET 语言与标准库层面的可落地技术**来写，目标是“能读懂工程代码、能写出惯用写法、面试能讲清楚”。一些跨语言也通用的设计理念（例如 OOP 原则、组合/继承取舍、分层与架构等）会以**短段落**穿插在相关语法点后面，作为使用提醒；更系统的“理念与模式”会另起笔记整理。

后文结构：

第 1 章 基础语法与通用编程能力

本章旨在建立 C# 编程的基本表达能力与调试习惯，内容涵盖变量与类型体系、表达式与运算符、分支与循环、方法与参数机制、基础输入输出与常用容器的入门使用，并配合断点调试、单步执行、调用栈与变量观察等手段完成问题定位。阶段性要求为能够独立完成若干控制台小程序，并具备将逻辑拆分为可复用函数的能力。

第 2 章 C# 语言特性与惯用写法

本章聚焦 C# 区别于通用语法层的核心特性与主流编码风格，重点包括属性与封装习惯、泛型与泛型集合、委托与事件模型、LINQ 数据查询表达、异常体系与资源管理（`IDisposable/using`）、以及异步编程模型（`async/await`）的基础认知，同时纳入类型安全与空值处理的基本规范。阶段性要求为能够读写具有典型 C# 风格的代码，并在小型模块中正确运用上述机制提升可读性与可维护性。

第 3 章 C# 项目构建（dotnet 工具链）

这一章专门补齐“从写代码到交付项目”的最小闭环：用 `dotnet` 命令行从 0 创建项目，理解 `.csproj` / `.sln` 在工程里的角色，掌握 NuGet 依赖管理的最小用法，并能区分 build/run/test/publish 的职责；如果走后端方向，还要能把一个最小 Web API 跑起来，顺手把 DI、配置、日志三件套接上。阶段性要求为能够独立创建一个带 `src/` 与 `tests/` 的小项目，跑通测试并产出可发布产物。

第 4 章 工程化能力与项目交付链路

本章面向“可维护、可测试、可发布”的工程目标，系统掌握 .NET 生态的项目组织与交付流程，包括解决方案与项目结构设计、NuGet 依赖管理、构建与发布（`dotnet` /MSBuild）、配置与环境区分、日志与错误边界治理、以及单元测试体系（xUnit/NUnit）与基础测试策略；同时将序列化、IO、异步与并发控制纳入工程约束进行综合训练。阶段性要求为能够产出结构清晰、具备测试、可稳定运行并可发布的完整小项目。

第 5 章 实战与作品集构建（方向实践）

本章以真实项目为载体完成能力整合：后端方向以 ASP.NET Core 为主线，覆盖 API 设计、中间件管线、鉴权、数据访问（EF Core）与部署；游戏方向以 Unity 为主线，覆盖组件化架构、生命周期管理、协程/异步、资源与场景管理、性能优化与网络交互。最终形成 2–3 个可展示项目，并以可运行、可发布、可扩展与可解释（技术取舍清晰）作为验收标准。

通用编程

基础格式

```
using System;
// using 关键字用于在程序中包含命名空间。一个程序可以包含多个 using 语句。
namespace RectangleApplication
// 命名空间，命名空间用于区分不同空间的名称定义，最常见的是用于分割不同文件间的名称定义
{
    class Rectangle
    // 类，一个类由成员变量与方法函数组成
    {
        // 成员变量
        double length;
        double width;
        // 成员函数
        public void Acceptdetails()
        {
            length = 4.5;
            width = 3.5;
        }
        public double GetArea()
        {
            return length * width;
        }
        public void Display()
        {
            Console.WriteLine("Length: {0}", length);
            Console.WriteLine("Width: {0}", width);
            Console.WriteLine("Area: {0}", GetArea());
        }
    }

    class ExecuteRectangle
    {
        // 执行入口，通常为Main函数
        static void Main(string[] args)
        {
            Rectangle r = new Rectangle();
            r.Acceptdetails();
            r.Display();
            Console.ReadLine();
        }
    }
}
```

预编译与命名空间

预编译

预处理器指令指导编译器在实际编译开始之前对信息进行预处理。通过这些指令，可以控制编译器如何编译文件或编译哪些部分。常见的预处理器指令包括条件编译、宏定义等。所有的预处理器指令都是以 **#** 开始，且在一行上，只有空白字符可以出现在预处理器指令之前。预处理器指令不是语句，所以它们不以分号 **;** 结束。C# 编译器没有一个单独的预处理器，但是，指令被处理时就像是有一个单独的预处理器一样。在 C# 中，预处理器指令用于在条件编译中起作用。与 C 和 C++ 不同的是，它们不是用来创建宏。一个预处理器指令必须是该行上的唯一指令。通过预编译可以实现：1. **提高代码可读性**：使用 `#region` 可以帮助分隔代码块，提高代码的组织性。2. **条件编译**：通过 `#if` 等指令可以在开发和生产环境中编译不同的代码，方便调试和发布。3. **警告和错误**：通过 `#warning` 和 `#error` 可以在编译时提示开发人员注意特定问题。

以下是C# 中可用的预处理器指令：

指令	描述
`#define`	定义一个符号，可以用于条件编译。
`#undef`	取消定义一个符号。
`#if`	开始一个条件编译块，如果符号被定义则包含代码块。
`#elif`	如果前面的 `#if` 或 `#elif` 条件不满足，且当前条件满足，则包含代码块。
`#else`	如果前面的 `#if` 或 `#elif` 条件不满足，则包含代码块。
`#endif`	结束一个条件编译块。
`#warning`	生成编译器警告信息。
`#error`	生成编译器错误信息。
`#region`	标记一段代码区域，可以在IDE中折叠和展开这段代码，便于代码的组织和阅读。
`#endregion`	结束一个代码区域。
`#line`	更改编译器输出中的行号和文件名，可以用于调试或生成工具的代码。
`#pragma`	用于给编译器发送特殊指令，例如禁用或恢复特定的警告。
`#nullable`	控制可空性上下文和注释，允许启用或禁用对可空引用类型的编译器检查。

#define 用于定义符号（通常用于条件编译），**#undef** 用于取消定义符号。这样，通过使用符号作为传递给 #if 指令的表达式，条件指令用于测试符号是否为真。如果为真，编译器会执行 **#if** 和下一个指令之间的代码。一个以 **#if** 指令开始的条件指令，必须显示地以一个 **#endif** 指令终止。

```
#define DEBUG

#if DEBUG
    Console.WriteLine("Debug mode");
#elif RELEASE
    Console.WriteLine("Release mode");
#else
    Console.WriteLine("Other mode");
#endif
```

#warning 用于生成编译器警告，**#error** 用于生成编译器错误。

```
#warning This is a warning message
#error This is an error message
```

#region 和 **#endregion** 用于代码折叠，使代码更加可读。**#line** 用于更改文件行号和文件名的编译器输出。**#pragma** 用于向编译器发送特殊指令。最常见的用法是禁用特定的警告。

```
#region MyRegion
    // Your code here
#endregion

#line 100 "MyFile.cs"
    // The next line will be reported as line 100 in MyFile.cs
    Console.WriteLine("This is line 100");
#line default
    // Line numbering returns to normal

#pragma warning disable 414
    private int unusedVariable;
#pragma warning restore 414
```

命名空间

命名空间的设计目的是提供一种让一组名称与其他名称分隔开的方式。在一个命名空间中声明的类的名称与另一个命名空间中声明的相同的类的名称不冲突。命名空间的定义是以关键字 **namespace** 开始，后跟命名空间的名称。为了调用支持命名空间版本的函数或变量，会把命名空间的名称置于前面。**using** 关键字表明程序使用的是给定命名空间中的名称。该指令告诉编译器随后的代码使用了指定命名空间中的名称。命名空间可以嵌套，使用点（.）运算符访问嵌套的命名空间的成员。

```
// 1.a 使用 using 引入命名空间
using System;
using SomeNameSpace;
using SomeNameSpace.Nested;
// 1.b 为命名空间起别名
using Nested = SomeNameSpace.Nested;

// 2. 定义一个命名空间
namespace SomeNameSpace
{
    public class MyClass
```

```
    {
        static void Main()
        {
            // 3.此时不需要使用完整的路径即可使用 System 下的函数
            Console.WriteLine("In SomeNameSpace");
            Nested.NestedNameSpaceClass.SayHello();
        }
    }

    // 4.内嵌命名空间
    namespace Nested
    {
        public class NestedNameSpaceClass
        {
            public static void SayHello()
            {
                Console.WriteLine("In Nested");
            }
        }
    }
}
```

类型、变量与常量

类型

变量类型：值类型、引用类型、指针类型

值类型：值类型变量可以直接分配给一个值。它们是从类 **System.ValueType** 中派生的。值类型直接包含数据。比如 **int**、**char**、**float**，它们分别存储数字、字符、浮点数。当您声明一个 **int** 类型时，系统分配内存来存储值。如需得到一个类型或一个变量在特定平台上的准确尺寸，可以使用 **sizeof** 方法。表达式 *sizeof(type)* 产生以字节为单位存储对象或类型的存储尺寸。

```
// 1. 使用sizeof获取数据类型长度
Console.WriteLine("Size of int: {0}", sizeof(int));
```

类型	描述	范围	默认值
bool	布尔值	True 或 False	False
byte	8 位无符号整数	0 到 255	0
char	16 位 Unicode 字符	U +0000 到 U +ffff	'\0'
decimal	128 位精确的十进制值，28-29 有效位数	(-7.9 x 1028 到 7.9 x 1028) / 100 到 28	0.0M
double	64 位双精度浮点型	(+/-)5.0 x 10-324 到 (+/-)1.7 x 10308	0.0D
float	32 位单精度浮点型	-3.4 x 1038 到 + 3.4 x 1038	0.0F
int	32 位有符号整数类型	-2,147,483,648 到 2,147,483,647	0

类型	描述	范围	默认值
long	64 位有符号整数类型	-9,223,372,036,854,775,808 到 9,223,372,036,854,775,807	0L
sbyte	8 位有符号整数类型	-128 到 127	0
short	16 位有符号整数类型	-32,768 到 32,767	0
uint	32 位无符号整数类型	0 到 4,294,967,295	0
ulong	64 位无符号整数类型	0 到 18,446,744,073,709,551,615	0
ushort	16 位无符号整数类型	0 到 65,535	0

引用类型：引用类型指的是一个内存位置，它不包含存储在变量中的实际数据，但它们包含对变量的引用。如果内存位置的数据是由一个变量改变的，其他变量会自动反映这种值的变化。**内置的** 引用类型有：**object**、**dynamic** 和 **string**。

```
// 1.object 类型
object obj;
obj = 100; // 这是装箱
// 2. dynamic 类型
dynamic d = 20; // 运行时推测类型
// 3. string 类型
string str = @"C:\Windows"; // 逐字处理""内的字符
string str = "C:\\Windows"; // 转义处理\后的字符
string str = @"<script type=""text/javascript""">
    <!—
    —>
</script>"; //逐字符后允许随意换行表示字符串
```

类型	描述
对象（Object）类型	C# 通用类型系统（CTS）中所有数据类型的终极基类。Object 是 System.Object 类的别名。所以对象（Object）类型可以被分配任何其他类型（值类型、引用类型、预定义类型或用户自定义类型）的值。但是，在分配值之前，需要先进行类型转换。当一个值类型转换为对象类型时，则被称为 **装箱**；另一方面，当一个对象类型转换为值类型时，则被称为 **拆箱**。
动态（Dynamic）类型	可以存储任何类型的值在动态数据类型变量中。这些变量的类型检查是在**运行时**发生的。
字符串（String）类型	字符串（String）类型是 System.String 类的别名。它是从对象（Object）类型派生的。字符串（String）类型的值可以通过两种形式进行分配：引号和 @引号。
自定义引用类型	class、interface 或 delegate等是可以自定义的引用类型。

指针类型：指针类型变量存储内存地址（仅在 `unsafe` 代码中使用）。它和“引用类型的引用”不是一回事：引用类型是运行时可跟踪的托管引用（对象可被 GC 移动/回收），而指针是原生地址，编译器与运行时不会为你提供同等的安全保证。

类型转换：将一个数据类型的值转换为另一个数据类型的过程。可以分为两种：**隐式类型转换**和**显式类型转换**（也称为强制类型转换）。隐式转换是不需要编写代码来指定的转换，编译器会自动进行。例如，从 int 到 long，从 float 到 double 等。从小的整数类型转换为大的整数类型，从派生类转换为基类。将一个 byte 类型的变量赋值给 int 类型的变量，编译器会自动将 byte 类型转换为 int 类型，不需要显示转换。显式类型转换，即强制类型转换，需要程序员在代码中明确指定，强制转换会造成数据丢失。本身有许多方法提供了类型转换，例如ToType()，Convert，Parse。

```
// 0. 强制转换
double doubleValue = 3.14;
int intValue = (int)doubleValue; // 强制从 double 到 int，数据可能损失小数部分
// 1. Convert 类提供了一组静态方法，可以在各种基本数据类型之间进行转换。
string str = "123";
int num = Convert.ToInt32(str);
// 2. Parse 方法用于将字符串转换为对应的数值类型，如果转换失败会抛出异常。
string str = "123.45";
double d = double.Parse(str);
// 3. TryParse 方法类似于 Parse，但它不会抛出异常，而是返回一个布尔值指示转换是否成功。
string str = "123.45";
double d;
bool success = double.TryParse(str, out d);

if (success) {
    Console.WriteLine("转换成功: " + d);
} else {
    Console.WriteLine("转换失败");
}

// 4. C# 还允许你定义自定义类型转换操作，通过在类型中定义 implicit 或 explicit 关键字。
using System;
public class Fahrenheit
{
    public double Degrees { get; set; }

    public Fahrenheit(double degrees)
    {
        Degrees = degrees;
    }

    // 隐式转换从Fahrenheit到double
    public static implicit operator double(Fahrenheit f)
    {
        return f.Degrees;
    }

    // 显式转换从double到Fahrenheit
    public static explicit operator Fahrenheit(double d)
    {
        return new Fahrenheit(d);
    }
```

```
}
public class Program
{
    public static void Main()
    {
        Fahrenheit f = new Fahrenheit(98.6);
        Console.WriteLine("Fahrenheit object: " + f.Degrees + " degrees");

        double temp = f; // 隐式转换
        Console.WriteLine("After implicit conversion to double: " + temp + " degrees");

        Fahrenheit newF = (Fahrenheit)temp; // 显式转换
        Console.WriteLine("After explicit conversion back to Fahrenheit: " + newF.Degrees + " degrees");
    }
}
```

变量

变量定义：`<data_type> <variable_list>;` 在这里，data_type 必须是一个有效的 C# 数据类型，可以是 char、int、float、double 或其他用户自定义的数据类型。variable_list 可以由一个或多个用逗号分隔的标识符名称组成。变量的命名需要遵循一些规则：1. 变量名可以包含字母、数字和下划线。2. 变量名必须以字母或下划线开头。3. 变量名区分大小写。4. 避免使用 C# 的关键字作为变量名。

变量作用域：1. 在方法、循环、条件语句等代码块内声明的变量是局部变量，它们只在声明它们的代码块中可见。2. 块级作用域，即使用大括号 **{}** 创建的任何块都可以定义变量的作用域。3. 方法的参数也有其自己的作用域，它们在整个方法中都是可见的。4. 在类的成员级别定义的变量是成员变量，它们在整个类中可见，如果在命名空间级别定义，那么它们在整个命名空间中可见。5. 静态变量是在类级别上声明的，但它们的作用域也受限于其定义的类。

常量

常量是固定值，程序执行期间不会改变。常量可以是任何基本数据类型，比如整数常量、浮点常量、字符常量或者字符串常量，还有枚举常量。常量可以被当作常规的变量，只是它们的值在定义后不能被修改。常量是使用 **const** 关键字来定义的 。

运算符与表达式

运算符

运算符可以分为**算术运算符、关系运算符、逻辑运算符、位运算符、赋值运算符、其他运算符**

运算符	描述	实例
+	把两个操作数相加	A + B 将得到 30
-	从第一个操作数中减去第二个操作数	A - B 将得到 -10
*	把两个操作数相乘	A * B 将得到 200
/	分子除以分母	B / A 将得到 2
%	取模运算符，整除后的余数	B % A 将得到 0
++	自增运算符，整数值增加 1	A++ 将得到 11

运算符	描述	实例
--	自减运算符，整数值减少 1	A-- 将得到 9
==	检查两个操作数的值是否相等，如果相等则条件为真。	(A == B) 不为真。
!=	检查两个操作数的值是否相等，如果不相等则条件为真。	(A != B) 为真。
>	检查左操作数的值是否大于右操作数的值，如果是则条件为真。	(A > B) 不为真。
<	检查左操作数的值是否小于右操作数的值，如果是则条件为真。	(A < B) 为真。
>=	检查左操作数的值是否大于或等于右操作数的值，如果是则条件为真。	(A >= B) 不为真。
<=	检查左操作数的值是否小于或等于右操作数的值，如果是则条件为真。	(A <= B) 为真。
&&	逻辑与（短路与）。两个操作数都为 true 时结果为 true；若左侧为 false 则右侧不会计算（短路）。	(A && B) 为假。
||	逻辑或（短路或）。任一操作数为 true 时结果为 true；若左侧为 true 则右侧不会计算（短路）。	(A || B) 为真。
!	称为逻辑非运算符。用来逆转操作数的逻辑状态。如果条件为真则逻辑非运算符将使其为假。	!(A && B) 为真
&	如果同时存在于两个操作数中，二进制 AND 运算符复制一位到结果中。	(A & B) 将得到 12，即为 0000 1100
|	如果存在于任一操作数中，二进制 OR 运算符复制一位到结果中。	(A | B) 将得到 61，即为 0011 1101
^	如果存在于其中一个操作数中但不同时存在于两个操作数中，二进制异或运算符复制一位到结果中。	(A ^ B) 将得到 49，即为 0011 0001
~	按位取反运算符是一元运算符，具有"翻转"位效果，即0变成1，1变成0，包括符号位。	(~A) 将得到 -61，即为 1100 0011，一个有符号二进制数的补码形式。
<<	二进制左移运算符。左操作数的值向左移动右操作数指定的位数。	A << 2 将得到 240，即为 1111 0000
>>	二进制右移运算符。左操作数的值向右移动右操作数指定的位数。	A >> 2 将得到 15，即为 0000 1111

运算符	描述	实例
=	简单的赋值运算符，把右边操作数的值赋给左边操作数	C = A + B 将把 A + B 的值赋给 C
+=	加且赋值运算符，把右边操作数加上左边操作数的结果赋值给左边操作数	C += A 相当于 C = C + A
-=	减且赋值运算符，把左边操作数减去右边操作数的结果赋值给左边操作数	C -= A 相当于 C = C - A
*=	乘且赋值运算符，把右边操作数乘以左边操作数的结果赋值给左边操作数	C *= A 相当于 C = C * A
/=	除且赋值运算符，把左边操作数除以右边操作数的结果赋值给左边操作数	C /= A 相当于 C = C / A
%=	求模且赋值运算符，求两个操作数的模赋值给左边操作数	C %= A 相当于 C = C % A
<<=	左移且赋值运算符	C <<= 2 等同于 C = C << 2
>>=	右移且赋值运算符	C >>= 2 等同于 C = C >> 2
&=	按位与且赋值运算符	C &= 2 等同于 C = C & 2
^=	按位异或且赋值运算符	C ^= 2 等同于 C = C ^ 2
|=	按位或且赋值运算符	C |= 2 等同于 C = C | 2
sizeof()	返回数据类型的大小。	sizeof(int)，将返回 4.
typeof()	返回 class 的类型。	typeof(StreamReader);
&	返回变量的地址。	&a; 将得到变量的实际地址。
*	变量的指针。	*a; 将指向一个变量。
? :	条件表达式	如果条件为真 ? 则为 X : 否则为 Y
is	判断对象是否为某一类型。	If(Ford is Car) // 检查 Ford 是否是 Car 类的一个对象。
as	强制转换，即使转换失败也不会抛出异常。	Object obj = new StringReader("Hello"); StringReader r = obj as StringReader;

优先级如下：

类别	运算符	结合性
后缀	() [] -> . ++ - -	从左到右
一元	+ - ! ~ ++ - - (type)* & sizeof	从右到左
乘除	* / %	从左到右

类别	运算符	结合性
加减	+ -	从左到右
移位	<< >>	从左到右
关系	< <= > >=	从左到右
相等	== !=	从左到右
位与 AND	&	从左到右
位异或 XOR	^	从左到右
位或 OR	|	从左到右
逻辑与 AND	&&	从左到右
逻辑或 OR	||	从左到右
条件	?:	从右到左
赋值	= += -= *= /= %=>>= <<= &= ^= |=	从右到左
逗号	,	从左到右

数组

数组是一个存储相同类型元素的固定大小的顺序集合。数组中某个指定的元素是通过索引来访问的。所有的数组都是由连续的内存位置组成的。最低的地址对应第一个元素，最高的地址对应最后一个元素。

其使用形式如下：

```
// 声明数组
double[] balance;
// 初始化数组
double[] balance = new double[10];
double[] balance = { 2340.0, 4523.69, 3421.0};
// 访问数组
double salary = balance[9];
for ( i = 0; i < 10; i++ )
{
    balance[i] = i + 100;
}
foreach (double j in balance )
{
    int i = j-100;
    Console.WriteLine("Element[{0}] = {1}", i, j);
}
```

字符串

string 关键字是 **System.String** 类的别名,使用 **string** 关键字来声明一个字符串变量。

其使用形式如下:

```
using System;

namespace StringApplication
{
    class Program
    {
        static void Main(string[] args)
        {
           //字符串，字符串连接
            string fname, lname;
            fname = "Rowan";
            lname = "Atkinson";

            string fullname = fname + lname;
            Console.WriteLine("Full Name: {0}", fullname);

            //通过使用 string 构造函数
            char[] letters = { 'H', 'e', 'l', 'l','o' };
            string greetings = new string(letters);
            Console.WriteLine("Greetings: {0}", greetings);

            //方法返回字符串
            string[] sarray = { "Hello", "From", "Tutorials", "Point" };
            string message = String.Join(" ", sarray);
            Console.WriteLine("Message: {0}", message);

            //用于转化值的格式化方法
            DateTime waiting = new DateTime(2012, 10, 10, 17, 58, 1);
            string chat = String.Format("Message sent at {0:t} on {0:D}",
            waiting);
            Console.WriteLine("Message: {0}", chat);
            Console.ReadKey() ;
        }
    }
}
```

String 类有以下两个属性:

序号	属性名称 & 描述
1	**Chars** 在当前 *String* 对象中获取 *Char* 对象的指定位置。
2	**Length** 在当前的 *String* 对象中获取字符数。

String 类有许多方法用于 string 对象的操作。下面的表格提供了一些最常用的方法:

序号	方法名称 & 描述
1	**public static int Compare(string strA, string strB)** 比较两个指定的 string 对象，并返回一个表示它们在排列顺序中相对位置的整数。该方法区分大小写。
2	**public static int Compare(string strA, string strB, bool ignoreCase)** 比较两个指定的 string 对象，并返回一个表示它们在排列顺序中相对位置的整数。但是，如果布尔参数为真时，该方法不区分大小写。
3	**public static string Concat(string str0, string str1)** 连接两个 string 对象。
4	**public static string Concat(string str0, string str1, string str2)** 连接三个 string 对象。
5	**public static string Concat(string str0, string str1, string str2, string str3)** 连接四个 string 对象。
6	**public bool Contains(string value)** 返回一个表示指定 string 对象是否出现在字符串中的值。
7	**public static string Copy(string str)** 创建一个与指定字符串具有相同值的新的 String 对象。
8	**public void CopyTo(int sourceIndex, char[] destination, int destinationIndex, int count)** 从 string 对象的指定位置开始复制指定数量的字符到 Unicode 字符数组中的指定位置。
9	**public bool EndsWith(string value)** 判断 string 对象的结尾是否匹配指定的字符串。
10	**public bool Equals(string value)** 判断当前的 string 对象是否与指定的 string 对象具有相同的值。
11	**public static bool Equals(string a, string b)** 判断两个指定的 string 对象是否具有相同的值。
12	**public static string Format(string format, Object arg0)** 把指定字符串中一个或多个格式项替换为指定对象的字符串表示形式。
13	**public int IndexOf(char value)** 返回指定 Unicode 字符在当前字符串中第一次出现的索引，索引从 0 开始。
14	**public int IndexOf(string value)** 返回指定字符串在该实例中第一次出现的索引，索引从 0 开始。
15	**public int IndexOf(char value, int startIndex)** 返回指定 Unicode 字符从该字符串中指定字符位置开始搜索第一次出现的索引，索引从 0 开始。
16	**public int IndexOf(string value, int startIndex)** 返回指定字符串从该实例中指定字符位置开始搜索第一次出现的索引，索引从 0 开始。
17	**public int IndexOfAny(char[] anyOf)** 返回某一个指定的 Unicode 字符数组中任意字符在该实例中第一次出现的索引，索引从 0 开始。
18	**public int IndexOfAny(char[] anyOf, int startIndex)** 返回某一个指定的 Unicode 字符数组中任意字符从该实例中指定字符位置开始搜索第一次出现的索引，索引从 0 开始。
19	**public string Insert(int startIndex, string value)** 返回一个新的字符串，其中，指定的字符串被插入在当前 string 对象的指定索引位置。
20	**public static bool IsNullOrEmpty(string value)** 指示指定的字符串是否为 null 或者是否为一个空的字符串。

序号	方法名称 & 描述
21	**public static string Join(string separator, string[] value)** 连接一个字符串数组中的所有元素，使用指定的分隔符分隔每个元素。
22	**public static string Join(string separator, string[] value, int startIndex, int count)** 连接一个字符串数组中的指定位置开始的指定元素，使用指定的分隔符分隔每个元素。
23	**public int LastIndexOf(char value)** 返回指定 Unicode 字符在当前 string 对象中最后一次出现的索引位置，索引从 0 开始。
24	**public int LastIndexOf(string value)** 返回指定字符串在当前 string 对象中最后一次出现的索引位置，索引从 0 开始。
25	**public string Remove(int startIndex)** 移除当前实例中的所有字符，从指定位置开始，一直到最后一个位置为止，并返回字符串。
26	**public string Remove(int startIndex, int count)** 从当前字符串的指定位置开始移除指定数量的字符，并返回字符串。
27	**public string Replace(char oldChar, char newChar)** 把当前 string 对象中，所有指定的 Unicode 字符替换为另一个指定的 Unicode 字符，并返回新的字符串。
28	**public string Replace(string oldValue, string newValue)** 把当前 string 对象中，所有指定的字符串替换为另一个指定的字符串，并返回新的字符串。
29	**public string[] Split(params char[] separator)** 返回一个字符串数组，包含当前的 string 对象中的子字符串，子字符串是使用指定的 Unicode 字符数组中的元素进行分隔的。
30	**public string[] Split(char[] separator, int count)** 返回一个字符串数组，包含当前的 string 对象中的子字符串，子字符串是使用指定的 Unicode 字符数组中的元素进行分隔的。int 参数指定要返回的子字符串的最大数目。
31	**public bool StartsWith(string value)** 判断字符串实例的开头是否匹配指定的字符串。
32	**public char[] ToCharArray()** 返回一个带有当前 string 对象中所有字符的 Unicode 字符数组。
33	**public char[] ToCharArray(int startIndex, int length)** 返回一个带有当前 string 对象中所有字符的 Unicode 字符数组，从指定的索引开始，直到指定的长度为止。
34	**public string ToLower()** 把字符串转换为小写并返回。
35	**public string ToUpper()** 把字符串转换为大写并返回。
36	**public string Trim()** 移除当前 String 对象中的所有前导空白字符和后置空白字符。

结构体

*结构体（struct）是一种值类型（value type），用于组织和存储相关数据。**struct** 关键字用于创建结构体。*

引用类型对象本身通常分配在托管堆上；值类型（struct）的“存储位置”取决于它出现在哪里：作为局部变量时往往在栈上（或被优化到寄存器），作为引用类型字段或数组元素时会跟随其宿主而在堆上；被装箱时也会在堆上。工程上选择 struct 的理由主要是“值语义 + 小而稳定 + 可避免额外分配”，而不是简单的“栈更快”。如果你希望共享同一实例并通过引用修改，或者对象较大/有复杂生命周期，通常用 class 更合适。

其使用形式如下：

```
using System;
using System.Text;

struct Books
{
   public string title;
   public string author;
   public string subject;
   public int book_id;
};

public class testStructure
{
   public static void Main(string[] args)
   {

      Books Book1;        /* 声明 Book1，类型为 Books */
      Books Book2;        /* 声明 Book2，类型为 Books */

      /* book 1 详述 */
      Book1.title = "C Programming";
      Book1.author = "Nuha Ali";
      Book1.subject = "C Programming Tutorial";
      Book1.book_id = 6495407;

      /* book 2 详述 */
      Book2.title = "Telecom Billing";
      Book2.author = "Zara Ali";
      Book2.subject =  "Telecom Billing Tutorial";
      Book2.book_id = 6495700;

      /* 打印 Book1 信息 */
      Console.WriteLine( "Book 1 title : {0}", Book1.title);
      Console.WriteLine("Book 1 author : {0}", Book1.author);
      Console.WriteLine("Book 1 subject : {0}", Book1.subject);
      Console.WriteLine("Book 1 book_id :{0}", Book1.book_id);

      /* 打印 Book2 信息 */
      Console.WriteLine("Book 2 title : {0}", Book2.title);
      Console.WriteLine("Book 2 author : {0}", Book2.author);
      Console.WriteLine("Book 2 subject : {0}", Book2.subject);
      Console.WriteLine("Book 2 book_id : {0}", Book2.book_id);

      Console.ReadKey();

   }
```

```
}
```

枚举类型

枚举是一组命名整型常量。枚举类型是使用 **enum** *关键字声明的。**C# 枚举是值类型。换句话说，枚举包含自己的值，且不能继承或传递继承。*

其使用形式如下：

```
enum <enum_name>
{
    enumeration list
};

using System;

public class EnumTest
{
    enum Day { Sun, Mon, Tue, Wed, Thu, Fri, Sat };

    static void Main()
    {
        int x = (int)Day.Sun;
        int y = (int)Day.Fri;
        Console.WriteLine("Sun = {0}", x);
        Console.WriteLine("Fri = {0}", y);
    }
}
```

语句

判断语句

语句	描述
if 语句	一个 **if 语句** 由一个布尔表达式后跟一个或多个语句组成。
if...else 语句	一个 **if 语句** 后可跟一个可选的 **else 语句**，else 语句在布尔表达式为假时执行。
嵌套 if 语句	您可以在一个 **if** 或 **else if** 语句内使用另一个 **if** 或 **else if** 语句。
switch 语句	一个 **switch** 语句允许测试一个变量等于多个值时的情况。
嵌套 switch 语句	您可以在一个 **switch** 语句内使用另一个 **switch** 语句。
C# Null 条件运算符	C# 6.0 引入了 Null 条件运算符 **?.**，用于安全地访问属性或方法，而不必写一堆 if 判断。

语句	描述
? : 运算符	Exp1 ? Exp2 : Exp3; ? 表达式的值是由 Exp1 决定的。如果 Exp1 为真，则计算 Exp2 的值，结果即为整个 ? 表达式的值。如果 Exp1 为假，则计算 Exp3 的值，结果即为整个 ? 表达式的值。

```
// 1. if / if else / if 嵌套语句
if(boolean_expression)
{
   /* 如果布尔表达式为真将执行的语句 */
}
else
{
  /* 如果布尔表达式为假将执行的语句 */
}
// 2. switch / 嵌套 switch 语句
switch(ch1)
{
   case 'A':
      Console.WriteLine("这个 A 是外部 switch 的一部分");
      switch(ch2)
      {
         case 'A':
            Console.WriteLine("这个 A 是内部 switch 的一部分");
            break;
         case 'B': /* 内部 B case 代码 */
            break;
         default:
            break;
      }
      break;
   case 'B': /* 外部 B case 代码 */
      break;
   default:
      break;
}
// 3. 三目运算符
Exp1 ? Exp2 : Exp3;
```

C# Null 条件运算符:Null 条件运算符是 C# 6.0 引入的一个语法糖，它允许开发者在访问对象的成员或元素之前，先检查该对象是否为 `null` 。如果对象为 `null` ，则整个表达式的结果为 `null` ，而不会抛出 `NullReferenceException` 。简单来说就是在访问前加?，如果为空则停止访问。

语法	说明
`a?.b`	如果 `a` 不为 null，则访问 `a.b` ；否则返回 null
`a?.Method()`	如果 `a` 不为 null，则调用方法；否则什么都不做
`a?[i]`	如果 `a` 不为 null，则访问索引；否则返回 null
`?`	可空类型声明

语法	说明
`??`	Null 合并运算符
`?.`	Null 条件运算符
`?[]`	Null 条件索引运算符
`?:`	条件（三元）运算符

循环语句

循环类型	描述
while 循环	当给定条件为真时，重复语句或语句组。它会在执行循环主体之前测试条件。
for/foreach 循环	多次执行一个语句序列，简化管理循环变量的代码。
do...while 循环	除了它是在循环主体结尾测试条件外，其他与 while 语句类似。
嵌套循环	您可以在 while、for 或 do..while 循环内使用一个或多个循环。

控制语句	描述
break 语句	终止 **loop** 或 **switch** 语句，程序流将继续执行紧接着 loop 或 switch 的下一条语句。
continue 语句	跳过本轮循环，开始下一轮循环。

```
using System;

namespace Loops
{

    class Program
    {
        static void Main(string[] args)
        {
            /* 局部变量定义 */
            int a = 10;

            /* while 循环执行 */
            while (a < 20)
            {
                Console.WriteLine("a 的值:  {0}", a);
                a++;
                if (a > 15)
                {
                    /* 使用 break 语句终止 loop */
                    break;
                }
            }
            Console.ReadLine();
```

```

            /* 局部变量定义 */
            int b = 10;

            /* do 循环执行 */
            do
            {
                if (b == 15)
                {
                    /* 跳过迭代 */
                    b = b + 1;
                    continue;
                }
                Console.WriteLine("b 的值:  {0}", b);
                b++;

            } while (b < 20);

            Console.ReadLine();
        }
    }
}
```

异常与异常处理

异常处理

异常是在程序执行期间出现的问题。C# 中的异常是对程序运行时出现的特殊情况的一种响应，比如尝试除以零。异常提供了一种把程序控制权从某个部分转移到另一个部分的方式。C# 异常处理时建立在四个关键词之上的：**try**、**catch**、**finally** 和 **throw**。**try**块标识了一个将被激活的特定的异常的代码块，后跟一个或多个 catch 块。**catch**块在程序通过异常处理程序捕获异常。**finally**块用于执行给定的语句，不管异常是否被抛出都会执行。**throw**块用于在问题出现时，抛出一个异常。

```
using System;
namespace ErrorHandlingApplication
{
    class DivNumbers
    {
        int result;
        DivNumbers()
        {
            result = 0;
        }
        public void division(int num1, int num2)
        {
            try
            {
                result = num1 / num2;
            }
            catch (DivideByZeroException e)
            {
                Console.WriteLine("Exception caught: {0}", e);
```

```
            }
            finally
            {
                Console.WriteLine("Result: {0}", result);
            }

        }
        static void Main(string[] args)
        {
            DivNumbers d = new DivNumbers();
            d.division(25, 0);
            Console.ReadKey();
        }
    }
}
```

异常

C# 异常是使用类来表示的。C# 中的异常类主要是直接或间接地派生于 **System.Exception** 类。**System.ApplicationException** 和 **System.SystemException** 类是派生于 System.Exception 类的异常类。**System.ApplicationException** 曾经用于区分“应用异常”和“系统异常”，但在现代 .NET 实践中，自定义异常一般直接继承 `System.Exception` （或更具体的异常基类），并在需要时通过 `InnerException` 保留原始异常。**System.SystemException** 类是所有预定义的系统异常的基类。如果异常是直接或间接派生自 **System.Exception** 类，则可以抛出一个对象。可以在 catch 块中使用 throw 语句来抛出当前的对象。

下表列出了一些派生自 System.SystemException 类的预定义的异常类：

异常类	描述
System.IO.IOException	处理 I/O 错误。
System.IndexOutOfRangeException	处理当方法指向超出范围的数组索引时生成的错误。
System.ArrayTypeMismatchException	处理当数组类型不匹配时生成的错误。
System.NullReferenceException	处理当依从一个空对象时生成的错误。
System.DivideByZeroException	处理当除以零时生成的错误。
System.InvalidCastException	处理在类型转换期间生成的错误。
System.OutOfMemoryException	处理空闲内存不足生成的错误。
System.StackOverflowException	处理栈溢出生成的错误。

面向对象

封装

封装这一块读到工程代码时，除了访问修饰符、字段/属性这种“经典 OOP”内容，你还会频繁碰到一些更偏语言与工程习惯的延伸规则：比如空值标注（NRT）、不可变建模（readonly/init/private set）、资源释放（IDisposable/using）、以及 class/struct/record 的类型选型。它们确实和“把状态约束写进类型里”有关，但不属于 OOP 核心概念本身。这里先把主线留给字段与属性，以上内容放到后文统一整理。

字段与属性

字段是存储细节，属性是对外契约。你把状态“如何读、能不能写、写入要不要校验”都统一收敛到属性上，外部代码就只看见一致的入口（`s.Name`），而内部随时可以更换实现（字段、懒加载、计算属性、缓存等）。下面用表格把你最常见的封装写法拉平对照，看到源码就能快速定位作者的意图。

目标	常见写法	外部能做什么	内部能做什么	典型用途
只是存数据（最简单）	`public int Age { get; set; }`	读写	读写	DTO、非常轻的模型（不强调不变量）
构造后不允许变	`public int Id { get; }`	只读	构造时赋值	身份标识、创建时间等
外部只读、内部可控改	`public string Name { get; private set; }`	只读	通过方法按规则改	实体状态，配合 `Rename/Update ...`
初始化后冻结	`public string Major { get; init; }`	初始化阶段可赋值	初始化阶段可赋值	配置/DTO、一次性组装模型
计算属性	`public int Score => Math.Clamp(_score,0,100);`	只读（取计算结果）	内部可变字段/算法	派生值、格式化、只读视图
访问器里做校验/规范化	`set { if (...) throw; _x = value.Trim(); }`	通过属性写入	统一校验与修正	不变量维护（name 非空等）

示例（沿用学生系统实体，示例类型 + Main 操作演示）：

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public class Student
    {
```

```
        private string _name;
        private int _age;

        public int Id { get; }

        public string Name
        {
            get => _name;
            private set
            {
                if (string.IsNullOrWhiteSpace(value))
                    throw new ArgumentException("Name cannot be empty.", nameof(value));
                _name = value.Trim();
            }
        }

        public int Age
        {
            get => _age;
            private set
            {
                if (value < 0 || value > 150)
                    throw new ArgumentOutOfRangeException(nameof(value));
                _age = value;
            }
        }

        public Student(int id, string name, int age)
        {
            Id = id;
            Name = name; // 走属性校验
            Age = age;   // 走属性校验
        }

        public void Rename(string newName) => Name = newName;
        public void Birthday() => Age += 1;

        public override string ToString() => $"#{Id} {Name} ({Age})";
    }

    public static class Program
    {
        public static void Main()
        {
            var s = new Student(1, " Alice ", 18);
            Console.WriteLine(s);
            s.Rename("Alice Zhang");
            s.Birthday();
            Console.WriteLine(s);
        }
    }
}
```

方法与参数传递

方法签名最重要的信息是三件事：返回什么、需要什么参数、调用方能不能通过参数“把结果写回去”。C# 的参数传递里最常见的三种形式是值参数（默认）、引用参数（`ref`）、输出参数（`out`）。它们的差异用一句话总结就是：默认参数是“传值的副本”；`ref` 是“传同一个变量的引用（读写都共享）”；`out` 是“我只负责写出结果，你调用前不用初始化”。下面用表格把区别摆清楚，再用一个学生分数计算的小例子跑一遍。

方式	关键字	调用前变量要不要初始化	方法内部能做什么	调用后调用方变量会不会变	典型用途
值参数	无	✅	读写形参，但不影响实参	❌（实参不变）	绝大多数场景（安全、直觉）
引用参数	`ref`	✅	读写同一个变量	✅	需要“就地修改”或避免拷贝的大 struct
输出参数	`out`	❌	必须给 out 赋值后才能返回	✅	Try 模式：`TryParse/TryGet ...`

示例（示例类型 + Main 演示）：

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public static class ScoreMath
    {
        public static int ClampScore(int score)
        {
            if (score < 0) return 0;
            if (score > 100) return 100;
            return score;
        }

        public static void AddBonus(ref int score, int bonus)
        {
            score = ClampScore(score + bonus);
        }

        public static bool TryParseScore(string? text, out int score)
        {
            if (int.TryParse(text, out var s))
            {
                score = ClampScore(s);
                return true;
            }

            score = default; // out 参数必须赋值
            return false;
        }
```

```
    }

    public static class Program
    {
        public static void Main()
        {
            int a = 95;
            Console.WriteLine($"before ref: {a}");
            ScoreMath.AddBonus(ref a, 10);
            Console.WriteLine($"after ref: {a}");

            if (ScoreMath.TryParseScore("120", out var s1))
                Console.WriteLine($"parsed score: {s1}");

            if (!ScoreMath.TryParseScore("not-a-number", out var s2))
                Console.WriteLine($"parse failed, score default: {s2}");
        }
    }
}
```

构造链与继承初始化

构造函数的职责是把对象带入“合法状态”。当一个类有多个构造函数时，如果每个构造函数都重复做校验和赋值，很容易写出不一致的规则，因此常见做法是用 `this( ... )` 把多个构造汇聚到一个“主构造”，让校验只写一份。存在继承时，用 `base( ... )` 先初始化基类部分，保证基类不变量成立，再初始化派生类自己的字段。配合 `readonly` 字段与 `init` 属性，你可以把“只能初始化一次”的约束写进类型本身。下面用表格把这些关键字出现时代表的意图摆出来。

写法	你在代码里看到的样子	它表达的意图	常见用途
`this( ... )`	`public Student( ... ) : this( ... ) { }`	复用本类构造逻辑	多个重载构造统一校验与赋值
`base( ... )`	`public Student( ... ) : base( ... ) { }`	先保证基类合法	继承初始化，避免基类处于非法状态
`readonly` 字段	`private readonly DateTime _createdAt;`	构造后不再赋值	创建时间、关键依赖、不可变标记
`init` 属性	`public string Major { get; init; }`	初始化可写，创建后冻结	配置/DTO 或派生类可选字段

示例（示例类型 + Main 操作演示）：

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public class Person
    {
```

```
        public int Id { get; }
        public string Name { get; protected set; }

        public Person(int id, string name)
        {
            ArgumentNullException.ThrowIfNull(name);
            Id = id;
            Name = name.Trim();
        }

        public override string ToString() => $"#{Id} {Name}";
    }

    public class Student : Person
    {
        private readonly DateTime _createdAt;

        public int Age { get; private set; }
        public string Major { get; init; } = "Undeclared";

        public Student(int id, string name, int age) : base(id, name)
        {
            _createdAt = DateTime.UtcNow;
            SetAge(age);
        }

        // this(...): 把“默认年龄”的规则收敛到主构造里
        public Student(int id, string name) : this(id, name, age: 18) { }

        public void SetAge(int age)
        {
            if (age < 0 || age > 150) throw new ArgumentOutOfRangeException(nameof(age));
            Age = age;
        }

        public override string ToString()
            => $"{base.ToString()} ({Age}) Major={Major} Created={_createdAt:O}";
    }

    public static class Program
    {
        public static void Main()
        {
            var s1 = new Student(1, "Alice", 20) { Major = "CS" };
            var s2 = new Student(2, "Bob") { Major = "Math" };
            Console.WriteLine(s1);
            Console.WriteLine(s2);
        }
    }
}
```

继承

接口 vs 抽象类

接口和抽象类都能支撑多态，但它们解决的痛点不一样。接口更像“边界契约”，重点是让调用方只依赖一组能力（方法签名），从而把实现替换成本压到最低；抽象类更像“模板骨架”，重点是把多个实现共同遵守的一套流程集中起来（比如：保存前要规范化、加载后要校验），子类只负责补齐少数扩展点。工程里更稳的习惯是：先用接口把边界立起来，等到你真的发现“多个实现都在重复同一段流程”时，再引入抽象类承载骨架；如果一开始就用抽象类当边界，调用方往往会被继承层级捆住，替换与测试会更重。

面临的问题	更像哪种需求	优先选	在 StudentManagerApp 里的常见落点
我只关心“能存/能取”，实现可能换（内存/文件/数据库）	能力契约、可替换	`interface`	`IStudentStore`：StudentManager 只依赖它
多种存储实现都要做同样的“流程”：保存前去重/规范化，加载后再规范化	共享流程骨架	`abstract class`	`StudentStoreBase`：把 Save/Load 的固定步骤写死
我需要强制某些点必须由子类实现（否则没法工作）	必须实现的扩展点	`abstract` 成员	`SaveCore/LoadCore` 必须提供
我只是想复用代码，但不想建立 IS-A	更适合组合	组合优先	StudentManager “持有 store”，不“继承 store”

下面把示例改成更像学生管理系统的真实用法：StudentManager 只依赖接口；内存存储与文件存储都可替换；抽象基类负责“保存/加载都要做 Normalize”的骨架；Main 里演示同一套管理逻辑换一个 store 就能跑。

```
#nullable enable
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;

namespace StudentManagerApp
{
    public sealed class Student
    {
        public int Id { get; }
        public string Name { get; }
        public Student(int id, string name)
        {
            Id = id;
            Name = string.IsNullOrWhiteSpace(name) ? throw new ArgumentException(nameof(name)) : name.Trim();
        }
        public override string ToString() => $"#{Id} {Name}";
    }

    public interface IStudentStore
```

```
22      {
23          void Save(List<Student> students);
24          List<Student> Load();
25      }
26
27      public abstract class StudentStoreBase : IStudentStore
28      {
29          public void Save(List<Student> students)
30          {
31              ArgumentNullException.ThrowIfNull(students);
32              var normalized = Normalize(students);
33              SaveCore(normalized);
34          }
35
36          public List<Student> Load()
37          {
38              var loaded = LoadCore();
39              return Normalize(loaded);
40          }
41
42          // 这里就是“骨架里固定的一步”：把重复规则放在这里，所有实现共享
43          protected virtual List<Student> Normalize(List<Student> students)
44          {
45              // 示例规则：按 Id 去重，Name 再 Trim（真实项目按业务调整）
46              return students
47                  .Where(s => s is not null)
48                  .GroupBy(s => s.Id)
49                  .Select(g => g.Last())
50                  .ToList();
51          }
52
53          protected abstract void SaveCore(List<Student> students);
54          protected abstract List<Student> LoadCore();
55      }
56
57      public sealed class MemoryStudentStore : StudentStoreBase
58      {
59          private readonly List<Student> _cache = new();
60
61          protected override void SaveCore(List<Student> students)
62          {
63              _cache.Clear();
64              _cache.AddRange(students);
65          }
66
67          protected override List<Student> LoadCore()
68          {
69              return new List<Student>(_cache);
70          }
71      }
72
73      public sealed class FileStudentStore : StudentStoreBase
74      {
```

```
        private readonly string _path;

        public FileStudentStore(string path)
        {
            _path = string.IsNullOrWhiteSpace(path) ? throw new ArgumentException(nameof(path)) : path;
        }

        protected override void SaveCore(List<Student> students)
        {
            using var writer = new StreamWriter(File.Open(_path, FileMode.Create, FileAccess.Write, FileShare.Read));
            foreach (var s in students)
                writer.WriteLine($"{s.Id},{s.Name}");
        }

        protected override List<Student> LoadCore()
        {
            if (!File.Exists(_path)) return new List<Student>();

            var list = new List<Student>();
            foreach (var line in File.ReadAllLines(_path))
            {
                var parts = line.Split(',', 2);
                if (parts.Length == 2 && int.TryParse(parts[0], out var id))
                    list.Add(new Student(id, parts[1]));
            }
            return list;
        }
    }

    public sealed class StudentManager
    {
        private readonly IStudentStore _store;
        private readonly List<Student> _students = new();

        public StudentManager(IStudentStore store) => _store = store;

        public void Add(Student s) => _students.Add(s);

        public void Save() => _store.Save(_students);

        public void Load()
        {
            _students.Clear();
            _students.AddRange(_store.Load());
        }

        public void Print()
        {
            foreach (var s in _students) Console.WriteLine(s);
        }
    }
```

```

    public static class Program
    {
        public static void Main()
        {
            IStudentStore store1 = new MemoryStudentStore();
            var mgr1 = new StudentManager(store1);

            mgr1.Add(new Student(1, "Alice"));
            mgr1.Add(new Student(1, "Alice (dup)"));
            mgr1.Add(new Student(2, "Bob"));
            mgr1.Save();

            var mgr2 = new StudentManager(store1);
            mgr2.Load();
            Console.WriteLine("== Load from MemoryStudentStore ==");
            mgr2.Print();

            IStudentStore store2 = new FileStudentStore("students.csv");
            var mgr3 = new StudentManager(store2);
            mgr3.Add(new Student(3, "Cathy"));
            mgr3.Add(new Student(4, "David"));
            mgr3.Save();

            var mgr4 = new StudentManager(store2);
            mgr4.Load();
            Console.WriteLine("== Load from FileStudentStore ==");
            mgr4.Print();
        }
    }
}
```

显式接口实现

显式接口实现主要是在两种情况下出现：一种是你同时实现多个接口，而这些接口恰好有同名成员（例如都叫 `Print()` ），这时需要“把同名方法分成两份”；另一种是你希望某些方法只在“接口视角”下暴露，类本身的 public API 里不出现它们。显式实现的直觉是：这不是给对象增加一个 public 方法，而是在告诉编译器“当你把我当作某个接口时，我应该怎么做”。因此调用时也会更显式：你必须先把对象转换成接口，再去调用那个成员。

你想解决的事	典型写法	调用方式	读代码时的直觉
两个接口成员同名（冲突）	`void IPrintable.Print()` + `void ILoggable.Print()`	`((IPrintable)obj).Print()`	把同名成员拆成两套实现
控制暴露面（不想 public 出现）	显式实现接口成员	必须转接口才能调	类的 API 更干净，能力被收进“边界视角”

下面把例子改成学生管理语境：报告既能“打印给用户看”，也能“输出到日志”，两种 Print 语义不同，用显式实现把它们分开，同时保留一个 `Preview()` 作为类自己的 public 行为。

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public interface IPrintable { void Print(); }
    public interface ILoggable { void Print(); }

    public sealed class StudentReport : IPrintable, ILoggable
    {
        private readonly string _title;

        public StudentReport(string title) => _title = title;

        void IPrintable.Print() => Console.WriteLine($"[PRINT] {_title}");
        void ILoggable.Print() => Console.WriteLine($"[LOG]  {_title}");

        public void Preview() => Console.WriteLine($"Preview: {_title}");
    }

    public static class Program
    {
        public static void Main()
        {
            var report = new StudentReport("Semester Report");
            report.Preview();

            // report.Print(); // 这里调不到，因为是显式实现
            ((IPrintable)report).Print();
            ((ILoggable)report).Print();
        }
    }
}
```

继承的边界

继承表达 IS-A，但工程里更怕的是“为了复用写继承”，因为继承把状态、行为和扩展点都绑在一起，层级一深就难维护。更常见的姿势是组合：把可替换的行为放到字段/属性里（通常是接口），需要换策略就换实现。确实需要运行时多态扩展点时，C# 真正的动态分派只发生在 `virtual/override`（或 `abstract/override`）上：基类必须把成员标成可重写，子类用 override 才会在“基类引用指向子类对象”时生效。`new` 只是隐藏同名成员，它更像是“子类自己额外提供了一个同名方法”，并不参与虚派发，所以会出现“看似写了 Describe，但通过基类引用调用时完全没走到”的坑。

关键字	直觉含义	运行时会不会按真实类型分派	工程里最常见的用途/坑
`virtual`	给子类留扩展点（可重写）	✅	基类提供默认实现
`override`	真正重写（多态生效）	✅	子类替换基类行为
`abstract`	必须重写（没有默认实现）	✅	明确“你必须实现”
`new`	隐藏同名成员（不是重写）	❌	典型坑：基类引用不会走 new

下面用学生管理里更容易理解的例子：`Student` 的描述是多态扩展点（override），而 `ExchangeStudent` 额外写了一个 `new Describe()` 来演示坑：当你把它当作 `Person` 使用时，仍然只会走 override 链，而不会走 new 的那份。

C# 语言特性

特性

特性就是“贴在代码上的描述信息”。它并不自动执行逻辑，更像在类型系统上额外贴了一张标签：类、方法、属性等元素上除了代码本身，还能附加一段可查询的元数据。真正的行为发生在“读特性的一方”：可能是编译器（例如 `[Obsolete]` 触发警告）、可能是运行库、也可能是框架或你自己写的反射扫描器。你看到一个 `[Something(...)]`，直觉上把它当成“这里声明了一个规则/约定”，然后去找“谁在读它、读到之后做了什么”。

你在代码里看到的样子	它在表达什么	典型用途	StudentManager 的直觉例子
`[Obsolete(...)]`	这个 API 旧了，不建议继续依赖	兼容性提示/迁移指引	`AddStudentV1(...)` 标为废弃，提示改用新方法
`[Conditional("DEBUG")]`	某些调用点只在特定编译符号下存在	调试日志/埋点	Debug 下打印更详细的添加/删除日志
`[AttributeUsage(...)]`	约束自定义特性可以贴在哪里、能否重复、是否继承	让特性不被乱贴	`[StudentColumn]` 只能贴属性上，避免贴到类/方法导致歧义
`[StudentColumn("age", ...)]`	这是“映射信息”，把外部列名映射到属性	CSV/JSON/表头映射	导入时按列名把字段写进 Student
`[Range(0,150, ...)]`	这是“校验规则”，表达合法范围	数据合法性约束	Age 超范围时在导入阶段直接报错

特性的参数来源也需要顺手看懂：`(...)` 里的“定位参数”来自特性构造函数；`Xxx = value` 这种“命名参数”来自可写的 public 属性/字段。另一个常见的语法糖是：特性类型名如果以 `Attribute` 结尾，那么在 `[...]` 这个上下文里可以省略后缀，所以 `[StudentColumn(...)]` 等价于 `[StudentColumnAttribute(...)]`，但这并不是全局重命名，在普通代码里你依然要写完整的类型名。

预定义特性

预定义特性很多，但对“读工程代码”最有帮助的常见三种是 `AttributeUsage`、`Conditional`、`Obsolete`。它们的价值都在于：你一看到标记，就能预判它会怎样影响调用方、编译器或运行时的行为。

特性	它在解决什么问题	你需要记住的行为	StudentManager 场景
`AttributeUsage`	防止自定义特性被乱贴、乱用	限制目标（Class/Method/Property...）、是否允许重复、是否继承	让 `[StudentColumn]` 只能贴属性上，且一个属性最多贴一次
`Conditional`	让“调用点”在编译期就被删掉	没定义符号时，对这个方法的调用语句会被移除（不是运行时 if）	DebugLog 在 Release 下调用行消失
`Obsolete`	给旧 API 加“迁移提示”	引用时给警告或错误；可强制错误	标记旧的导入接口 `LoadFromCsvV1()`

```
#nullable enable
using System;
using System.Diagnostics;

namespace StudentManagerApp
{
    public sealed class Student
    {
        public string Id { get; set; } = "";
        public string Name { get; set; } = "";
    }

    public sealed class StudentManager
    {
        [Conditional("DEBUG")]
        public static void DebugLog(string msg) => Console.WriteLine("[DEBUG] " + msg);

        [Obsolete("请改用 AddStudent(Student student)。该方法仅为兼容旧代码保留。", isError: false)]
        public void AddStudentV1(string id, string name)
        {
            DebugLog($"AddStudentV1 called: id={id}, name={name}");
            AddStudent(new Student { Id = id, Name = name });
        }
```

```

        public void AddStudent(Student student)
        {
            DebugLog($"AddStudent called: {student.Id} {student.Name}");
        }
    }
}
```

自定义特性

自定义特性就是一个继承 `System.Attribute` 的普通类。它最重要的意义在于：你可以把“规则写在类型上”，让规则靠近数据模型，而不是散落在各种导入/导出/校验函数里。这样字段一增一改，通常只需要改模型上的标注与属性声明，反射侧的“通用执行器”不必跟着改。

设计点	你会怎么写	直觉理解
继承与命名	`sealed class XxxAttribute : Attribute`	`Attribute` 后缀是约定，标注时可省略
定位参数	构造函数参数	必填信息放这里，例如列名
命名参数	public 可写属性/字段	可选信息放这里，例如 DisplayName、Message
使用约束	`[AttributeUsage(AttributeTargets.Property, ...)]`	把“能贴哪”写死，减少误用

下面这组特性用来表达 Student 的“列名映射”与“范围校验”。`StudentColumnAttribute` 负责描述导入/导出用的列名与必填性，`RangeAttribute` 负责描述数值范围与提示信息。

```
#nullable enable
using System;

namespace StudentManagerApp
{
    [AttributeUsage(AttributeTargets.Property, AllowMultiple = false, Inherited = true)]
    public sealed class StudentColumnAttribute : Attribute
    {
        public string ColumnName { get; }
        public bool Required { get; set; } = false;
        public string? DisplayName { get; set; }

        public StudentColumnAttribute(string columnName) => ColumnName = columnName;
    }

    [AttributeUsage(AttributeTargets.Property, AllowMultiple = false, Inherited = true)]
    public sealed class RangeAttribute : Attribute
    {
        public int Min { get; }
        public int Max { get; }
```

```
        public string? Message { get; set; }

        public RangeAttribute(int min, int max) { Min = min; Max = max; }
    }

    public sealed class Student
    {
        [StudentColumn("id", Required = true, DisplayName = "学号")]
        public string Id { get; set; } = "";

        [StudentColumn("name", Required = true, DisplayName = "姓名")]
        public string Name { get; set; } = "";

        [StudentColumn("age", DisplayName = "年龄")]
        [Range(0, 150, Message = "年龄必须在 0~150 之间")]
        public int Age { get; set; }
    }
}
```

反射

反射的重点不是“到处用反射”，而是让你能理解：当类型在编译期未知或需要通用化处理时，运行时仍能读取类型结构、读取特性、再按规则去做事情。框架常用反射做依赖注入扫描、序列化映射、MVC 路由、ORM 映射等；现代工程也会用源生成器（Source Generator）把一部分反射前移到编译期，但“反射在干什么”这件事仍然是读源码与理解框架行为的基础。

反射对象	你从哪里拿到	常用方法	你通常用它做什么
`Type`	`typeof(T)` / `obj.GetType()`	`GetProperties()` / `GetMethods()`	拿成员列表、读取特性
`PropertyInfo`	`type.GetProperties()`	`GetValue()` / `SetValue()`	通用读写属性
`Attribute`	`GetCustomAttribute<T>()`	读取特性实例与参数	把“规则”拿出来执行
`Activator`	`Activator.CreateInstance(type)`	-	动态创建对象
`MethodInfo`	`type.GetMethods()`	`Invoke(obj, args)`	动态调用（插件/命令路由）

本章把反射聚焦在一条主线：读取 Student 属性上的 `StudentColumnAttribute` 与 `RangeAttribute`，把一行数据（例如 CSV 解析后得到的 `Dictionary<string,string>`）映射到 Student，并在映射过程中完成必填校验与范围校验。这样你新增字段时更多是在“模型层加标注”，而不是到处改导入逻辑。

```
#nullable enable
using System;
using System.Collections.Generic;
using System.Reflection;
```

```
5
6   namespace StudentManagerApp
7   {
8       public static class StudentMapper
9       {
10          public static Student MapFrom(Dictionary<string, string> row)
11          {
12              var student = new Student();
13              Type t = typeof(Student);
14
15              foreach (var prop in t.GetProperties(BindingFlags.Public |
    BindingFlags.Instance))
16              {
17                  var col = prop.GetCustomAttribute<StudentColumnAttribute>();
18                  if (col is null) continue;
19
20                  string key = col.ColumnName;
21                  bool has = row.TryGetValue(key, out string? raw);
22
23                  if (col.Required && !has)
24                  {
25                      string show = col.DisplayName ?? prop.Name;
26                      throw new ArgumentException($"缺少必填字段：{show}（列名：{key}）");
27                  }
28
29                  if (!has) continue;
30
31                  object value;
32                  if (prop.PropertyType == typeof(string))
33                  {
34                      value = raw ?? "";
35                  }
36                  else if (prop.PropertyType == typeof(int))
37                  {
38                      if (!int.TryParse(raw, out int n))
39                          throw new ArgumentException($"字段 {key} 需要 int，但得到：{raw}");
40                      value = n;
41                  }
42                  else
43                  {
44                      throw new NotSupportedException($"暂不支持类型：
    {prop.PropertyType.Name}");
45                  }
46
47                  var range = prop.GetCustomAttribute<RangeAttribute>();
48                  if (range ≠ null && value is int iv)
49                  {
50                      if (iv < range.Min || iv > range.Max)
51                          throw new ArgumentException(range.Message ?? $"{prop.Name} 超出范围
    {range.Min}~{range.Max}");
52                  }
53
54                  prop.SetValue(student, value);
```

```
            }

            return student;
        }
    }

    public static class Demo
    {
        public static void Main()
        {
            var row = new Dictionary<string, string>
            {
                ["id"] = "S001",
                ["name"] = "Alice",
                ["age"] = "20"
            };

            var s = StudentMapper.MapFrom(row);
            Console.WriteLine($"{s.Id} {s.Name} Age={s.Age}");
        }
    }
}
```

属性

属性看起来像字段，但它更像“读写规则”。你写 `student.Name = ...` 实际执行的是 `set` 中的逻辑，你读 `student.Name` 实际执行的是 `get` 中的逻辑，所以属性是封装最核心的入口：校验、修剪、触发事件、只读控制、延迟计算，通常都通过属性访问器来承载。工程里一个很实用的直觉是：字段是实现细节，属性是对外契约；如果外部直接改字段，就绕过了所有规则，因此字段尽量私有，把外部访问统一收敛到属性上。

形态	写法	你什么时候用	StudentManager 直觉例子
自动属性	`public string Name { get; set; }`	纯存储、无规则	DTO/简单模型
私有 set	`public int Count { get; private set; }`	外部只读、内部可更新	Manager 的统计信息
只读属性	`public DateTime CreatedAt { get; }`	只初始化一次	Student 创建时间
自定义访问器	`set { if (...) throw; }`	需要校验/格式化	Id/Name 不允许空
计算属性	`public bool IsAdult => Age ≥ 18;`	从其他字段推导	是否成年

```
#nullable enable
using System;

```

```
namespace StudentManagerApp
{
    public sealed class Student
    {
        private string _id = "";
        private string _name = "";

        public string Id
        {
            get => _id;
            set
            {
                if (string.IsNullOrWhiteSpace(value)) throw new ArgumentException("Id 不能为空");
                _id = value.Trim();
            }
        }

        public string Name
        {
            get => _name;
            set
            {
                if (string.IsNullOrWhiteSpace(value)) throw new ArgumentException("Name 不能为空");
                _name = value.Trim();
            }
        }

        public int Age { get; set; }

        public DateTime CreatedAt { get; } = DateTime.UtcNow;

        public bool IsAdult => Age >= 18;

        public override string ToString() => $"{Id} {Name} Age={Age}";
    }
}
```

索引器

索引器就是“带参数的特殊属性”，用于把 `[]` 语法引入到自定义类型上。它固定写成 `this[ ... ]`，因为它本质上是“通过当前实例进行下标访问”，语法上没有名字可写；如果一个类型需要多种下标方式，就用不同的参数签名区分，例如 `this[string id]` 与 `this[int index]` 可以同时存在。索引器本身只是入口，找不到 key 时返回什么、是否抛异常、是否返回 null，这些都由你在 `get/set` 的代码里决定；交互型程序通常会额外提供 Try 风格方法，让调用方在不确定时走更安全的分支。

写法	访问语义	常见约定	StudentManager 的自然用法
`this[string id]`	按 key 访问	key 不存在可返回 null 或抛异常	`mgr["S001"]` 按学号找学生

写法	访问语义	常见约定	StudentManager 的自然用法
`this[int index]`	按序号访问	越界通常抛异常	`mgr[0]` 取列表第一个
只读索引器	只有 `get`	写入走方法以统一校验	Add/Remove 保证同步
读写索引器	`get/set`	常用于缓存/映射更新	`cache[key] = value`

```
#nullable enable
using System;
using System.Collections.Generic;

namespace StudentManagerApp
{
    public sealed class StudentManager
    {
        private readonly List<Student> _list = new();
        private readonly Dictionary<string, Student> _dict = new(StringComparer.OrdinalIgnoreCase);

        public int Count => _list.Count;

        public Student? this[string id] => _dict.TryGetValue(id, out var s) ? s : null;

        public Student this[int index] => _list[index];

        public bool AddStudent(Student s)
        {
            if (_dict.ContainsKey(s.Id)) return false;
            _dict.Add(s.Id, s);
            _list.Add(s);
            return true;
        }

        public bool RemoveStudent(string id)
        {
            if (!_dict.TryGetValue(id, out var s)) return false;
            _dict.Remove(id);
            _list.Remove(s);
            return true;
        }
    }
}
```

委托与事件

委托解决“把方法当作值传递/组合调用链”，事件解决“把通知通道变成可控的发布-订阅机制”。两者看起来都用 `+=/-=`，但语义不同：委托变量只要你拿到引用就能直接调用；事件则把触发权锁在声明事件的类内部，外部只能订阅/退订，不能 `=` 覆盖整条调用链，也不能直接触发，这样发布者/订阅者的职责边界就清晰了。工程上最常见签名是 `EventHandler` / `EventHandler<TEventArgs>`，它把 sender 与 args 固定为一致形式，读代码时辨识度很高。

对比点	委托	事件
本质	函数引用/回调	委托 + 访问限制（发布-订阅）
组合	`+=` 形成多播链，`-=` 移除	订阅也用 `+=/-=`，但外部不能 `=` 覆盖
触发权	持有者可调用	只能在声明事件的类型内部触发
常用类型	`Action` / `Func` / `Predicate<T>`	`EventHandler` / `EventHandler<TEventArgs>`

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public sealed class StudentChangedEventArgs : EventArgs
    {
        public Student Student { get; }
        public string Action { get; }
        public StudentChangedEventArgs(Student student, string action) { Student = student; Action = action; }
    }

    public partial class StudentManager
    {
        public event EventHandler<StudentChangedEventArgs>? StudentChanged;

        protected virtual void OnStudentChanged(Student s, string action)
            => StudentChanged?.Invoke(this, new StudentChangedEventArgs(s, action));

        public bool AddStudent(Student s, Func<Student, bool>? rule = null)
        {
            if (rule != null && !rule(s)) return false;
            if (_dict.ContainsKey(s.Id)) return false;

            _dict.Add(s.Id, s);
            _list.Add(s);

            OnStudentChanged(s, "Add");
            return true;
        }
    }
}
```

匿名方法与 Lambda

匿名方法与 Lambda 都是在“现场创建委托目标”的写法。它们最常出现在两类地方：一类是事件订阅，另一类是把规则/策略作为参数传入通用方法。Lambda 是现代主流写法，表达式形式更短更贴近业务意图；匿名方法 `delegate( ... ) { ... }` 在旧代码里更常见，它的优势在于更显式地强调“这里是一段委托块”。闭包是这类写法的自然副作用：lambda 捕获外部变量时，这些变量的生命周期会延长到委托不再被引用为止，因此在事件订阅中要注意退订，否则可能导致对象被长期引用而无法回收。

写法	示例	适用点
匿名方法	`delegate(Student s) { return s.Age ≥ 18; }`	旧代码常见，强调“这里是委托块”
Lambda 表达式	`s ⇒ s.Age ≥ 18`	最常用，短且可读
Lambda 语句块	`s ⇒ { Console.WriteLine(s); return true; }`	需要多行逻辑/临时变量
闭包	`int min=18; s ⇒ s.Age ≥ min`	规则依赖外部变量（注意生命周期）

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public static class Demo
    {
        public static void Main()
        {
            var mgr = new StudentManager();

            mgr.StudentChanged += (sender, e) ⇒
            {
                Console.WriteLine($"[Event] {e.Action}: {e.Student}");
            };

            Func<Student, bool> ageRule = delegate (Student s)
            {
                return s.Age ≥ 0 && s.Age ≤ 150;
            };

            Func<Student, bool> mustHaveName = s ⇒ !string.IsNullOrWhiteSpace(s.Name);

            mgr.AddStudent(new Student { Id = "S001", Name = "Alice", Age = 20 }, rule: mustHaveName);
            mgr.AddStudent(new Student { Id = "S002", Name = "Bob", Age = 999 }, rule: ageRule);
        }
    }
}
```

集合

StudentManager 里最常见的集合组合是：用 `List<Student>` 负责顺序、遍历、排序与展示，用 `Dictionary<string, Student>` 负责按学号快速查找。只要你同时维护两套结构，就需要把“增删改”集中在少数方法里，确保同步；查找尽量用 `TryGetValue` 表达“找不到是预期分支”，避免用异常当控制流。

集合	常用成员	差异点	StudentManager 落地习惯
`List<T>`	`Add/Remove/RemoveAt/Sort/Count/foreach`	访问按序号，越界会异常	用于展示与排序
`Dictionary<TKey,TValue>`	`TryGetValue/ContainsKey/Add/Remove`	`dict[key]` key 不存在会异常	用于按 Id 快查

```csharp
#nullable enable
using System;

namespace StudentManagerApp
{
    public partial class StudentManager
    {
        public bool TryGetStudent(string id, out Student? student)
            => _dict.TryGetValue(id, out student);

        public bool UpdateName(string id, string newName)
        {
            if (!_dict.TryGetValue(id, out var s)) return false;
            s.Name = newName;
            OnStudentChanged(s, "Update");
            return true;
        }

        public void PrintAll()
        {
            foreach (var s in _list) Console.WriteLine(s);
        }
    }
}
```

泛型

泛型的核心是“把类型当参数传入”。对工程实践最有用的地方是：集合、回调与事件都可以通过泛型变得既可复用又类型安全。相比 `object`，泛型减少强制转换，避免装箱/拆箱，也让 API 在编译期就能暴露错误。把泛型用在 StudentManager 里最自然的方式是：写一个通用的过滤工具，再由 Manager 复用它来查学生。

泛型形态	典型写法	你用它做什么	StudentManager 例子
泛型集合	`List<T>` / `Dictionary<TKey,TValue>`	存储与查找	`List<Student>`、`Dictionary<string, Student>`

泛型形态	典型写法	你用它做什么	StudentManager 例子
泛型委托	`Func<T, ... >` / `Action<T>`	规则/回调/策略	`Func<Student,bool>`
泛型事件	`EventHandler<TEventArgs>`	强类型事件参数	`EventHandler<StudentChangedEventArgs>`
泛型方法	`WhereToList<T>( ... )`	与类型无关的算法	通用过滤工具

```
#nullable enable
using System;
using System.Collections.Generic;

namespace StudentManagerApp
{
    public static class StudentFilters
    {
        public static List<T> WhereToList<T>(IEnumerable<T> source, Func<T, bool> predicate)
        {
            var result = new List<T>();
            foreach (var item in source)
                if (predicate(item)) result.Add(item);
            return result;
        }
    }

    public partial class StudentManager
    {
        public List<Student> Find(Func<Student, bool> predicate)
            => StudentFilters.WhereToList(_list, predicate);
    }
}
```

.NET 运行时与工程化

资源释放

在 .NET 里，“资源”不只是内存。内存由 GC 管，但很多东西 GC 管不了释放时机：文件句柄、Socket 连接、数据库连接、系统句柄、native 分配的内存……这些都属于“用完就该尽快归还”的资源。如果只靠 GC，你会遇到两个现实问题：释放时机不确定（对象什么时候回收由 GC 决定，你无法保证用完立刻释放），并且成本更高（带 Finalizer 的对象回收路径更重，会增加 GC 压力）。因此更实用的心智模型是：`Dispose()` 用来做确定性释放，让你主动决定“什么时候释放”；Finalizer（析构函数）只做最后兜底，解决“调用者忘记 Dispose”的极端情况，但它不保证及时，且更贵。所以工程上通常鼓励：只要类型持有需要显式释放的对象（例如 `Stream`、`StreamWriter`、`Timer`、连接类等），就实现 `IDisposable` 并让调用者用 `using`；只有当你真的直接持有非托管资源（句柄、native memory 等）时，才考虑引入 Finalizer 的完整模式。

把 `IDisposable` 写对的关键并不在于背模板，而在于把语义定清楚并保持一致。 `IDisposable` 也不是“必须继承某个基类才能主动释放”的机制，它只是一个接口，用来统一表达“这个对象需要被显式释放，并且释放入口叫 `Dispose()` ”。你当然可以自己写 `Close()` 或 `Release()` 来手动释放，但那样调用方就没法用 `using` ，也不容易和框架的通用习惯对齐。 `using` 之所以能“自动释放”，本质是编译器语法糖：它会把你的代码展开成 `try/finally` ，并在 `finally` 里调用 `Dispose()` ，从而保证就算中间抛异常也会释放资源。 `GC.SuppressFinalize(this)` 则需要放在正确的语境里理解：它不是“Dispose 必须写的咒语”，它解决的是“当你的类型存在 Finalizer 时，既然已经手动释放过了，就别再走 Finalizer 路径”，避免终结队列带来的额外成本；如果类型压根没有 Finalizer，这行写了不会坏，但也不是必须。

下面沿用学生管理系统的语境，用“写日志到文件”来展示最常见的场景：类型只持有需要释放的托管对象（ `StreamWriter` ），此时你只要在 `Dispose()` 里把它释放掉即可，不需要 Finalizer，也不需要完整的 `Dispose(bool disposing)` 模式。代码我分成两部分：示例类本身，以及 Main 里的示例操作，让你能直接看到 `using` 、手动 `Dispose` 、异常路径释放、以及 Dispose 后再用会发生什么。

```
using System;
using System.IO;

namespace StudentManagerApp
{
    // 示例代码：一个写日志的组件（持有 StreamWriter，需要 Dispose）
    public sealed class StudentLogWriter : IDisposable
    {
        private readonly StreamWriter _writer;
        private bool _disposed;

        public StudentLogWriter(string path)
        {
            _writer = new StreamWriter(File.Open(path, FileMode.Append, FileAccess.Write, FileShare.Read));
        }

        public void Write(string line)
        {
            if (_disposed) throw new ObjectDisposedException(nameof(StudentLogWriter));
            _writer.WriteLine(line);
            _writer.Flush();
        }

        public void Dispose()
        {
            if (_disposed) return; // 幂等：多次 Dispose 不出错
            _writer.Dispose();
            _disposed = true;

            // 这里没有 finalizer，所以这行不是必须；但写了也不会错
            GC.SuppressFinalize(this);
        }
    }

    // 示例操作代码：用 Main 展示 using / 手动 Dispose / 异常也会释放
    public static class Program
    {
```

```
38          public static void Main()
39          {
40              Console.WriteLine("═══ using block demo ═══");
41              using (var log1 = new StudentLogWriter("student.log"))
42              {
43                  log1.Write("Add student: Alice");
44                  log1.Write("Add student: Bob");
45              } // 到这里一定会 Dispose（即使上面抛异常）
46
47              Console.WriteLine("═══ using declaration demo ═══");
48              using var log2 = new StudentLogWriter("student.log");
49              log2.Write("Update student: Alice → Grade A");
50              // log2 会在 Main 结束前自动 Dispose（作用域末尾）
51
52              Console.WriteLine("═══ manual dispose demo (try/finally) ═══");
53              var log3 = new StudentLogWriter("student.log");
54              try
55              {
56                  log3.Write("Remove student: Bob");
57              }
58              finally
59              {
60                  log3.Dispose(); // 手动确保释放
61              }
62
63              Console.WriteLine("═══ disposed object demo ═══");
64              var log4 = new StudentLogWriter("student.log");
65              log4.Dispose();
66              try
67              {
68                  log4.Write("This will fail");
69              }
70              catch (ObjectDisposedException ex)
71              {
72                  Console.WriteLine($"Write after Dispose: {ex.GetType().Name}");
73              }
74          }
75      }
76  }
```

如果将来你确实需要直接管理非托管资源（native handle/native memory），才会引入包含 `Dispose(bool disposing)` + Finalizer 的完整模式，核心点依然是同一套语义：`Dispose()` 负责确定性释放，Finalizer 兜底，`GC.SuppressFinalize(this)` 避免重复终结成本。真实项目里如果你走到需要手写 `IntPtr + finalizer` 的程度，通常更稳妥的路线是优先使用 `SafeHandle` 或已有封装，把“可靠释放 + 兜底”交给框架来做，业务类尽量保持在“只需要 Dispose 托管字段”的简单层级，这样更不容易踩坑。

异常与错误处理

在 C# 里，异常是一套“把错误沿调用栈向上传递”的机制。它不是用来做正常分支的（例如“没查到学生”这种可预期情况更适合 `TryGet...` 或返回 `null`），但它非常适合表达“契约被破坏”或“当前操作无法继续”的场景，例如参数非法、文件损坏、网络失败、重复主键、对象已释放等。你读工程代码时经常会看到两条线并存：一条是用返回值表达可预期失败（例如 `bool TryAdd(...)`），另一条是用异常表达不该发生或无法恢复的错误（例如 `ArgumentException`、`InvalidOperationException`）。

`try/catch/finally` 的关键点在于 `finally`：它几乎等价于“无论如何都会执行”，用来保证收尾动作一定发生（释放资源、归还锁、恢复状态）。你前面看到 `using` 能“自动释放”，本质就是编译器把 `using` 展开成 `try/finally` 并在 `finally` 里调用 `Dispose()`，因此异常路径也能正确清理。`throw;` 与 `throw ex;` 的区别也很重要：`throw;` 会保留原始堆栈信息，而 `throw ex;` 会重置堆栈，调试时会把真正的出错位置“截断掉”；所以在“捕获后继续往外抛”的场景里一般用 `throw;`，除非你明确要替换成新的异常并用 `InnerException` 保存旧异常。

工程里更推荐把“异常抛在哪里”做得明确：入口处对参数做快速失败（例如 `ArgumentNullException.ThrowIfNull`），在真正的业务边界上捕获并转成可解释的结果（例如控制台/接口层打印友好信息），而在领域逻辑内部尽量让异常自然冒泡，避免层层吞掉细节。下面的表把你最常见的异常写法和语义放在一起，够你在面试里把“什么时候用异常”讲清楚。

写法/类型	你在代码里会看到什么	适合表达什么	StudentManager 里的直觉例子
`ArgumentNullException` / `ArgumentException` / `ArgumentOutOfRangeException`	入参检查、构造函数检查	调用方传了不合法参数，属于“契约被破坏”	`AddStudent(null)`、年龄 < 0
`InvalidOperationException`	当前状态不允许该操作	对象处于不该操作的状态	重复学号、已初始化两次
`IOException` / `FileNotFoundException`	文件/IO 失败	外部环境问题（磁盘、权限、路径）	读取学生文件失败
`ObjectDisposedException`	Dispose 后继续使用	生命周期错误	`logger.Dispose()` 后还写日志
`try/catch/finally`	捕获 + 清理	捕获异常并在 `finally` 做收尾	捕获导入错误但确保关闭文件
`throw;`	继续抛出同一个异常	保留堆栈	记录日志后把异常交给上层

```
using System;
using System.Collections.Generic;
using System.IO;

```

```
5   namespace StudentManagerApp
6   {
7       public sealed class StudentRepository
8       {
9           private readonly Dictionary<string, Student> _map =
    new(StringComparer.OrdinalIgnoreCase);
10
11          public void Add(Student s)
12          {
13              ArgumentNullException.ThrowIfNull(s);
14              if (string.IsNullOrWhiteSpace(s.Id)) throw new ArgumentException("Id 不能为空",
    nameof(s));
15              if (_map.ContainsKey(s.Id)) throw new InvalidOperationException($"学号重复:
    {s.Id}");
16              _map.Add(s.Id, s);
17          }
18
19          public Student? Find(string id)
20          {
21              if (string.IsNullOrWhiteSpace(id)) throw new ArgumentException("id 不能为空",
    nameof(id));
22              return _map.TryGetValue(id, out var s) ? s : null;
23          }
24
25          public static List<Student> LoadFromFile(string path)
26          {
27              if (string.IsNullOrWhiteSpace(path)) throw new ArgumentException("path 不能为
    空", nameof(path));
28
29              var list = new List<Student>();
30              StreamReader? reader = null;
31
32              try
33              {
34                  reader = new StreamReader(path);
35                  string? line;
36                  while ((line = reader.ReadLine()) != null)
37                  {
38                      // 这里只做演示：id,name,age
39                      var parts = line.Split(',');
40                      if (parts.Length < 3) throw new FormatException($"行格式不正确:
    {line}");
41
42                      var s = new Student
43                      {
44                          Id = parts[0].Trim(),
45                          Name = parts[1].Trim(),
46                          Age = int.Parse(parts[2].Trim())
47                      };
48                      list.Add(s);
49                  }
50                  return list;
51              }
```

```
            catch (IOException ex)
            {
                // 这里可以做日志记录，然后把异常继续交给上层决定怎么处理
                Console.WriteLine("[IO ERROR] " + ex.Message);
                throw; // 保留堆栈信息
            }
            finally
            {
                // 无论成功还是失败都执行
                reader?.Dispose();
            }
        }
    }

    public static class Program
    {
        public static void Main()
        {
            var repo = new StudentRepository();

            try
            {
                repo.Add(new Student { Id = "S001", Name = "Alice", Age = 20 });
                repo.Add(new Student { Id = "S001", Name = "Bob", Age = 21 }); // 演示：重复学号
            }
            catch (InvalidOperationException ex)
            {
                Console.WriteLine("[业务错误] " + ex.Message);
            }

            try
            {
                var students = StudentRepository.LoadFromFile("students.csv"); // 演示：文件可能不存在
                Console.WriteLine("Loaded: " + students.Count);
            }
            catch (FileNotFoundException)
            {
                Console.WriteLine("[提示] students.csv 不存在，请先生成或检查路径");
            }
            catch (Exception ex)
            {
                // 最后兜底：未知异常不吞掉细节
                Console.WriteLine("[未处理异常] " + ex);
            }
        }
    }
}
```

LINQ

LINQ 的价值不是“炫技”，而是把常见的“筛选、投影、排序、分组、聚合”写成统一风格，并且与 `IEnumerable<T>` 紧密配合。你需要形成的直觉是：LINQ 的大多数操作都是“延迟执行”的，`Where/Select` 这类方法通常只是返回一个新的可枚举序列，直到你 `ToList()`、`ToArray()`、`Count()` 或 `foreach` 迭代时才真正跑起来；这也是为什么你在调试时看到 `var q = ...` 并不立刻执行，只有枚举时才执行。另一个常见坑是：`First()` 找不到会抛异常，`FirstOrDefault()` 找不到会返回默认值；`Single()` 要求“必须且只能有一个”，不满足就抛异常，这在面试题里很常被拿来考察。

需求	常用 LINQ	直觉	备注
过滤	`Where`	“留下满足条件的”	延迟执行
投影/选择字段	`Select`	“把元素变成别的形状”	常用来做 DTO/字符串
排序	`OrderBy/ThenBy`	“按键排序”	默认升序，降序用 `OrderByDescending`
是否存在	`Any`	“有没有一个满足”	通常比 `Count()>0` 更好
全部满足	`All`	“是不是都满足”	常配合校验
取一个	`First/FirstOrDefault`	“取第一个”	是否抛异常要记清楚
分组	`GroupBy`	“按键分桶”	结果是分组序列
聚合	`Sum/Average/Max/Min`	“算一个数”	空集合时要小心（可能抛异常）

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace StudentManagerApp
{
    public static class LinqDemo
    {
        public static void Run(List<Student> students)
        {
            var adults = students.Where(s => s.Age ≥ 18).ToList();
            var names = students.Select(s => s.Name).ToList();
            var top3 = students.OrderByDescending(s => s.Age).Take(3).ToList();

            var byAge = students.GroupBy(s => s.Age / 10 * 10); // 20/30/40 分桶
            foreach (var g in byAge)
                Console.WriteLine($"AgeBand {g.Key}s => {g.Count()} students");

            bool hasTeen = students.Any(s => s.Age is ≥ 13 and ≤ 19);
            Console.WriteLine("Has teen? " + hasTeen);

```

```
            var maybe = students.FirstOrDefault(s => s.Id == "S999"); // 找不到返回 null（Student 是 class）
            Console.WriteLine(maybe == null ? "S999 not found" : maybe.ToString());
        }
    }
}
```

异步编程

`async/await` 解决的核心问题是：当你要做 IO（文件、网络、数据库）这种“等待外部”的工作时，不要用线程傻等（阻塞），而是把控制权还给调用方，让当前线程去做别的事，等 IO 完成再继续执行后续逻辑。你在工程里最常见的形式是：方法返回 `Task` 或 `Task<T>`，内部用 `await` 等待异步操作；异常会被捕获并封装到 `Task` 里，最终在 `await` 处重新抛出，因此你仍然用 `try/catch` 在 `await` 外层捕获即可。另一个必须形成的习惯是避免在异步方法里用 `.Result` 或 `.Wait()` 去“强行同步”，这容易造成死锁或线程池阻塞，尤其是在 UI/ASP.NET 的上下文里。

你会看到的签名	适合什么	说明
`async Task`	异步但不返回值	调用方 `await` 等完成
`async Task<T>`	异步并返回结果	常用于加载、查询、计算
`async void`	只用于事件处理器	不能被 `await`，异常难处理
`CancellationToken`	取消	工程里常和超时/用户取消配合

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Threading;
using System.Threading.Tasks;

namespace StudentManagerApp
{
    public static class StudentFileStore
    {
        public static async Task SaveAsync(string path, List<Student> students, CancellationToken ct = default)
        {
            ArgumentNullException.ThrowIfNull(students);

            await using var fs = new FileStream(path, FileMode.Create, FileAccess.Write, FileShare.Read);
            await using var sw = new StreamWriter(fs);

            foreach (var s in students)
            {
                ct.ThrowIfCancellationRequested();
                await sw.WriteLineAsync($"{s.Id},{s.Name},{s.Age}");
            }
        }

```

```
        public static async Task<List<Student>> LoadAsync(string path, CancellationToken ct = default)
        {
            var list = new List<Student>();

            await using var fs = new FileStream(path, FileMode.Open, FileAccess.Read, FileShare.ReadWrite);
            using var sr = new StreamReader(fs);

            while (!sr.EndOfStream)
            {
                ct.ThrowIfCancellationRequested();

                var line = await sr.ReadLineAsync();
                if (line == null) break;

                var parts = line.Split(',');
                if (parts.Length < 3) continue;

                list.Add(new Student
                {
                    Id = parts[0].Trim(),
                    Name = parts[1].Trim(),
                    Age = int.Parse(parts[2].Trim())
                });
            }

            return list;
        }
    }

    public static class AsyncDemo
    {
        public static async Task RunAsync()
        {
            var students = new List<Student>
            {
                new Student { Id="S001", Name="Alice", Age=20 },
                new Student { Id="S002", Name="Bob", Age=18 }
            };

            var cts = new CancellationTokenSource();

            try
            {
                await StudentFileStore.SaveAsync("students_async.csv", students, cts.Token);
                var loaded = await StudentFileStore.LoadAsync("students_async.csv", cts.Token);
                Console.WriteLine("Async loaded: " + loaded.Count);
            }
            catch (OperationCanceledException)
            {
```

```
                Console.WriteLine("Canceled.");
            }
            catch (Exception ex)
            {
                Console.WriteLine("Async error: " + ex.Message);
            }
        }
    }

    public static class Program2
    {
        public static void Main()
        {
            // 控制台里演示 async：用 GetAwaiter().GetResult() 作为入口桥接
            // 真正工程里更推荐顶层语句或 async Main（C# 7.1+ 支持 async Task Main）
            AsyncDemo.RunAsync().GetAwaiter().GetResult();
        }
    }
}
```

集合接口与只读视图

你在笔记里已经用到了 `IReadOnlyList<T>`，它确实是一个接口，用来表达“我给你一个列表视图，你可以按下标读、可以枚举，但不能改我内部集合”。这是一种很典型的封装手段：内部仍然用 `List<T>` 存储（便于增删），对外只暴露 `IReadOnlyList<T>` 或 `IEnumerable<T>`（减少被外部随意修改导致的不变量破坏）。同理，字典也有 `IReadOnlyDictionary<TKey,TValue>` 这种只读接口。面试里经常会追问的是：为什么对外返回接口而不是具体类型，以及 `IEnumerable<T>` 的“只负责枚举”与 `ICollection<T>` 的“带 Count 与增删”的区别。

接口	你能做什么	常见用途	直觉
`IEnumerable<T>`	只能枚举（foreach）	最小暴露面，最通用	“我只让你看/遍历”
`IReadOnlyList<T>`	枚举 + `Count` + 下标读	对外只读列表	“像 List 但不给改”
`ICollection<T>`	枚举 + `Count` + 增删	通用可变集合	“可增删的集合抽象”
`IList<T>`	`ICollection` + 下标读写	需要随机访问与修改	“列表可改”
`IDictionary<K,V>`	Key/Value 增删查	映射	“可改字典”
`IReadOnlyDictionary<K,V>`	Key/Value 只读查	对外只读映射	“不给改的字典”

```
using System;
using System.Collections.Generic;

namespace StudentManagerApp
{
    public class StudentManager
```

```
    {
        private readonly List<Student> _students = new();

        // 对外只读：外部拿到的是视图，无法 Add/Remove
        public IReadOnlyList<Student> Students => _students;

        public void Add(Student s)
        {
            ArgumentNullException.ThrowIfNull(s);
            _students.Add(s);
        }
    }

    public static class ReadOnlyDemo
    {
        public static void Main()
        {
            var mgr = new StudentManager();
            mgr.Add(new Student { Id="S001", Name="Alice", Age=20 });

            IReadOnlyList<Student> view = mgr.Students;
            Console.WriteLine(view[0]);

            // view.Add( ... ) 编译不通过：只读接口没有 Add
        }
    }
}
```

可空引用

现代 C#（8+）推荐开启可空引用类型（Nullable Reference Types, NRT）。开启后，引用类型会被分成两类：`string` 这种写法表达“这里按设计不应该是 null”，而 `string?` 表达“这里允许是 null”。它更像一种“把以前写在脑子里/文档里的约定”强制写进类型系统：你不再只靠注释说“这个参数可能为 null”，而是用类型直接告诉编译器和读代码的人。编译器随后会帮你做大量静态检查：如果你把一个可能为 null 的值塞给 `string`，它会发出警告；如果你在没判空的情况下对 `string?` 访问成员，它也会提示你。这些警告不等于运行时异常，但它们能把“将来可能空引用”的 bug 提前暴露出来，让你在写代码时就把边界想清楚。

写 NRT 的时候有一条特别实用的主线：尽量把“可能为 null 的边界”挡在输入处（外部数据、解析、IO、用户输入、数据库返回值），一旦进入你的领域对象（比如 `StudentProfile` 这种代表业务状态的对象），就尽量保持它处于“合法且非 null 的状态”。这背后其实是一个更通用的建模思路：构造函数的职责是把对象初始化成可用状态，属性的类型是你对外承诺的约束；如果某个字段在业务上必须存在（比如学生的 Name），就用非可空类型并在构造入口做判空/清洗；如果某个字段在业务上就是可选的（比如 Nickname），就用 `string?` 并且在设置时把“空字符串/全空白”统一归一成 null，这样调用方的判断会更一致（只需要判断是否为 null，而不是同时处理空串、空白串、null 三种）。

在具体写法上，`ArgumentNullException.ThrowIfNull(x)` 是一种很推荐的入口判空方式，它表达得很直接：如果传进来的参数是 null，那是调用方的错误，你就立即抛异常而不是让对象处于半残状态。`??` 和 `??=` 则更像“输入修正”的工具：当某个来源允许缺省，你可以用它给出默认值（例如 `nickname ?? "Unknown"`），或者只在为空时赋默认（例如 `nickname ??= "Unknown"`）。至于 `!`（null-forgiving）这个符号，它的意思是“我向编译器保证这里一定不是 null，你别再警告我”，它能快速消除警告，但风险也最大：如果你的保证是错的，运行时照样会空引用，所以它更适合在你已经用别的方式证明过非空但编译器推断不到的少数场景里临时使用；大多数时候更好的选择是把判空写得更明确，让类型和代码一起把约束表达出来。

下面这段示例代码沿用学生管理系统：`Name` 是业务必填，所以用 `string`；`Nickname` 是可选，所以用 `string?`。构造函数负责把对象带入“合法状态”（判空、Trim、统一把空白昵称变成 null），方法入口也保持同样的规则，这样对象的内部状态就比较干净，后续使用时也更少需要到处判空。

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public class StudentProfile
    {
        public string Name { get; }
        public string? Nickname { get; private set; }

        public StudentProfile(string name, string? nickname = null)
        {
            ArgumentNullException.ThrowIfNull(name);
            Name = name.Trim();
            Nickname = string.IsNullOrWhiteSpace(nickname) ? null : nickname.Trim();
        }

        public void SetNickname(string? nickname)
        {
            Nickname = string.IsNullOrWhiteSpace(nickname) ? null : nickname.Trim();
        }
    }

    public static class Program
    {
        public static void Main()
        {
            // Name 是必填：传 null 会立刻抛 ArgumentNullException
            var alice = new StudentProfile(" Alice ", "  Ali  ");
            Console.WriteLine($"Name={alice.Name}, Nickname={alice.Nickname}");

            // Nickname 可选：空白/空串会被统一成 null
            alice.SetNickname("   ");
            Console.WriteLine($"After SetNickname: Nickname={(alice.Nickname is null ? "(null)" : alice.Nickname)}");

            // 典型的“输入边界”：外部数据可能为 null
            string? fromApi = GetNicknameFromExternal();
            // 用 ?? 给默认值，避免把 null 带进不允许为 null 的地方
            string safe = fromApi ?? "Unknown";
            Console.WriteLine($"safe nickname = {safe}");
        }

        private static string? GetNicknameFromExternal()
        {
            // 模拟外部数据：可能返回 null
            return null;
        }
    }
```

```
}
```

类型选型

在 C# 里，“对象”不只意味着 `class` 。你写一个类型时，其实是在做一个很关键的建模选择：这个东西在业务上是不是“有身份、有生命周期、会被多人引用同一个实例”？还是说它更像一个“纯数据/纯数值”，拿来传来传去只是表示一个值？选 `class` / `struct` / `record` 的核心差别就围绕这几个问题展开，语法细节反而是后面的事。最直观的分界线是“身份（identity）”：如果你在意的是“这是同一个学生对象”，而不是“这个学生现在有哪些字段值”，那它就更像实体，倾向用 `class` ；如果你在意的是“这个分数段是 60~79”，拿到另一个 `60~79` 你就认为它是同一个概念，那它更像值对象，倾向用 `struct` 或 `record` 。

`class` 是引用类型，默认语义是“共享同一实例”。把一个 `Student` 赋值给另一个变量只是复制引用，两个变量指向同一个对象；你在一处改了 `Name` ，另一处看到的也变了，这正符合“实体”的直觉。实体通常还会有生命周期：创建、修改、被仓储保存、被多个模块持有引用，因此 `class` 对这类场景很自然。与此同时，引用类型还有一个现实：它可能为 `null` ，所以在开启 NRT 的情况下你会更希望把领域对象尽量保持为非 null（构造时就保证合法），而把“可能为 null”的情况限制在输入边界。

`struct` 是值类型，更像“数据块”。赋值、传参会发生拷贝语义（除非被优化或通过 `ref` / `in` ），这让它很适合表达“很小、很稳定、拿来当值使用”的东西，比如坐标、范围、索引、金额这类值对象。值类型一个很大的优势是它默认不能为 null（除非用 `Nullable<T>` 也就是 `T?` ），因此在某些场景里能减少空值分支；但它也有代价：如果 struct 太大（字段很多）或者频繁复制，会带来拷贝成本，所以工程上常见建议是让 struct 保持“小而简单”，并且能的话用 `readonly struct` 让它更接近不可变值对象，避免被误修改导致额外拷贝与语义混乱。

`record` 则更像“为值语义而生的建模工具”。很多时候我们写的是“结构化数据”，我们希望它的相等性是“字段相同就算相等”，希望能方便地解构、打印、以及用 `with` 表达“基于旧值创建新值”。 `record` 默认就按值比较（而普通 class 默认按引用比较），并且会自动生成更友好的 `ToString()` 、解构等。这里有一个容易误解的点： `record` 并不等于“值类型”，因为 `record` 默认是 `record class` （引用类型）；它只是把“值语义”的默认行为带到了引用类型上，特别适合 DTO、配置、不可变模型等“按内容比较”的场景。如果你希望既是值语义又是值类型，还可以用 `record struct` ，它是值类型 record，适合高性能场景的值对象，但同样要注意不要做得太大。

放回学生管理系统来理解就很清楚： `Student` 通常是实体（因为它有 Id、有生命周期、会被仓储管理、会被多处共享同一实例），所以用 `class` 合适；“分数段/范围/索引”这种东西更像值对象，拿来传递时你只在意它的值，不在意它是不是同一个实例，所以用 `record` （强调值语义与不可变）或 `readonly struct` （强调值类型与轻量）更自然。你也会发现它们往往组合使用：实体里持有若干值对象，值对象帮助你把业务规则（比如分数段判断）封装得更干净。

下面的示例延续 StudentManagerApp，并且在 Main 里用一段小演示把“引用语义 vs 值语义 vs record 的按值比较”直观跑一遍，这样你不用靠背概念，而是能从输出里建立直觉。

```
#nullable enable
using System;

namespace StudentManagerApp
{
    // 实体：有身份（Id），通常被共享引用，生命周期清晰
    public class Student
    {
        public int Id { get; }
        public string Name { get; private set; }
```

```
        public Student(int id, string name)
        {
            ArgumentNullException.ThrowIfNull(name);
            Id = id;
            Name = name.Trim();
        }

        public void Rename(string name)
        {
            ArgumentNullException.ThrowIfNull(name);
            Name = name.Trim();
        }

        public override string ToString() => $"Student(Id={Id}, Name={Name})";
    }

    // 值对象：强调“内容相同就算相等”，适合不可变结构化数据
    public record ScoreRange(int Min, int Max)
    {
        public bool Contains(int score) => score ≥ Min && score ≤ Max;
        public override string ToString() => $"[{Min}..{Max}]";
    }

    // 小型值对象：更像“轻量数据块”，用 readonly struct 表达不可变 + 值语义
    public readonly struct StudentIndex
    {
        public int Value { get; }
        public StudentIndex(int value) => Value = value;
        public override string ToString() => Value.ToString();
    }

    public static class Program
    {
        public static void Main()
        {
            Console.WriteLine("=== class: reference semantics (shared instance) ===");
            var s1 = new Student(1, "Alice");
            var s2 = s1;              // 复制引用：s2 和 s1 指向同一个对象
            s2.Rename("Alice Zhang"); // 改 s2 等于改 s1
            Console.WriteLine($"s1 = {s1}");
            Console.WriteLine($"s2 = {s2}");
            Console.WriteLine($"ReferenceEquals(s1, s2) = {ReferenceEquals(s1, s2)}");
            Console.WriteLine();

            Console.WriteLine("=== record (record class): value-based equality by default ===");
            var r1 = new ScoreRange(60, 79);
            var r2 = new ScoreRange(60, 79);
            Console.WriteLine($"r1 = {r1}, r2 = {r2}");
            Console.WriteLine($"r1 == r2 : {r1 == r2} (same values => equal)");
            Console.WriteLine($"r1.Contains(75) = {r1.Contains(75)}");
            var r3 = r1 with { Max = 80 }; // 基于旧值生成新值（不可变风格很自然）
```

```
            Console.WriteLine($"r3 (with) = {r3}");
            Console.WriteLine();

            Console.WriteLine("== struct: value semantics (copy) ==");
            var i1 = new StudentIndex(42);
            var i2 = i1; // 拷贝值
            Console.WriteLine($"i1 = {i1}, i2 = {i2}");
            Console.WriteLine($"i1.Equals(i2) = {i1.Equals(i2)}");
        }
    }
}
```

值类型与引用类型、装箱拆箱与 ref/out/in

在 C# 里，“变量里装的到底是什么”会直接影响你写代码时的直觉：`class` 是引用类型，变量里存的是“指向对象的引用”；`struct` 是值类型，变量里通常直接存“数据本身”。这会导致两个最常见的差异：你把一个 `class` 变量赋给另一个变量，其实只是复制了引用，两边指向同一个对象；你把一个 `struct` 变量赋给另一个变量，是把整块数据拷贝了一份，之后各改各的互不影响。工程里值类型常用来表示“小、稳定、按值比较”的东西，例如分数区间、坐标、金额、学生成绩明细的某一项，而学生本体（有身份、可修改、会被多处引用）更适合用 `class`。

另外你会在面试和源码里频繁遇到“装箱/拆箱”和 `ref/out/in`。装箱（boxing）是把值类型当成 `object` 或接口来用时，运行时把它“包”成堆对象；拆箱（unboxing）则是把 `object` 里的值类型再取回来。它不是语法层面的事，而是运行时行为：会有分配与复制成本，并且拆箱类型不匹配会抛异常。`ref/out/in` 则是参数传递方式的显式声明：默认参数是“按值传递”，值类型会拷贝一份；`ref` 表示按引用传递（可读可写），`out` 表示“只负责输出”（方法内必须赋值），`in` 表示按引用传递但只读（主要用于避免大结构体拷贝）。理解它们后，你看 `TryParse` / `TryGetValue` / `Span<T>` 这类 API 就不会觉得“像魔法”。

主题	你需要的直觉	一句话记忆
`class`（引用类型）	复制的是引用，两变量指向同一实例	“同一个对象的两个遥控器”
`struct`（值类型）	复制的是值，会拷贝数据	“复印件，各改各的”
装箱/拆箱	值类型转 `object`/接口会分配；取回要类型匹配	“装箱会分配，拆箱要对号入座”
`ref`	传引用，可读可写	“把原件交给方法改”
`out`	传引用，方法必须先赋值再返回	“只出不进，必须写”
`in`	传引用但只读（避免拷贝）	“给你看原件但不许改”

```
using System;

namespace StudentManagerApp
{
    // 值类型：小而稳定的“成绩条目”，更像一个数据块
    public readonly struct ScoreEntry
    {
```

```
        public string Subject { get; }
        public int Score { get; }

        public ScoreEntry(string subject, int score)
        {
            Subject = string.IsNullOrWhiteSpace(subject) ? throw new ArgumentException(nameof(subject)) : subject.Trim();
            Score = score;
        }

        public override string ToString() => $"{Subject}:{Score}";
    }

    // 引用类型：学生是“有身份”的实体，会在多个地方被引用与更新
    public class Student
    {
        public string Id { get; set; } = "";
        public string Name { get; set; } = "";

        public override string ToString() => $"#{Id} {Name}";
    }

    public static class ValueRefDemo
    {
        public static void Run()
        {
            // 1) 引用类型：复制引用 ⇒ 改一边，另一边也看到
            var s1 = new Student { Id = "S001", Name = "Alice" };
            var s2 = s1;
            s2.Name = "Alice-Changed";
            Console.WriteLine($"s1.Name = {s1.Name}"); // Alice-Changed

            // 2) 值类型：复制值 ⇒ 改一边不影响另一边
            var e1 = new ScoreEntry("Math", 90);
            var e2 = e1;
            // e2.Score = 100; // readonly struct 不允许改（不可变值对象更安全）
            Console.WriteLine($"e1 = {e1}, e2 = {e2}");

            // 3) 装箱：值类型转 object（会发生分配）
            object boxed = e1;              // boxing
            var unboxed = (ScoreEntry)boxed; // unboxing（类型必须匹配）
            Console.WriteLine($"boxed type = {boxed.GetType().Name}, unboxed = {unboxed}");

            // 4) ref/out/in：Try 模式本质就是 out
            if (TryParseScore("English:88", out var parsed))
                Console.WriteLine("parsed ⇒ " + parsed);

            // 5) ref：交换两个值（演示“按引用改原变量”）
            int a = 1, b = 2;
            Swap(ref a, ref b);
            Console.WriteLine($"a={a}, b={b}");
        }
```

```
        private static bool TryParseScore(string text, out ScoreEntry entry)
        {
            entry = default;

            if (string.IsNullOrWhiteSpace(text)) return false;
            var parts = text.Split(':');
            if (parts.Length ≠ 2) return false;

            var subject = parts[0].Trim();
            if (!int.TryParse(parts[1], out int score)) return false;

            entry = new ScoreEntry(subject, score);
            return true;
        }

        private static void Swap(ref int x, ref int y)
        {
            int t = x;
            x = y;
            y = t;
        }
    }
}
```

不可变建模

封装做到熟练之后，你关注的就不只是“字段要不要 private”，而是“状态到底允许怎么变”：哪些状态一旦创建就不应该再变，哪些状态可以变但必须走特定入口，哪些状态只允许在初始化阶段写一次，哪些状态即便是集合也不能让外部拿到一个能随便改的引用。C# 给了你一组很直接的表达工具：只读属性、私有 set、init-only、readonly 字段，以及对集合只暴露只读视图或拷贝。它们的共同目标是把约束写进类型本身，让“能不能改、谁能改、什么时候能改”变成编译期可见的事实，而不是靠调用方自觉。

你想表达的约束	推荐写法（最常见）	外部能不能改	类内部能不能改	典型场景	备注/踩坑点
创建后永远不变（强不可变）	`public T P { get; }` + 构造函数赋值	❌	❌	值对象、规则对象、不可变状态	最容易推理；常配合 `record` 或 `readonly struct`
外部只读，内部可控改（半不可变）	`public T P { get; private set; }` + 行为方法 `Rename/Update ...`	❌	✅	实体（Student）、聚合根、需要生命周期的对象	“改状态只能走方法” = 校验/日志/事件都能集中管理
只允许初始化阶段赋值（初始化后冻结）	`public T P { get; init; }`	✅（仅初始化阶段）	✅（仅初始化阶段）	DTO、配置对象、一次性组装模型	对象创建完就不能再赋值；适合“组装后不再改”的数据
字段引用不变（但对象内容可能变）	`private readonly List<T> _list = new();`	外部不可见	✅	内部持有集合/依赖	`readonly` 只保证“字段不指向别处”，不保证集合内容不变
集合对外只读（防止外部 Add/Remove）	`public IReadOnlyList<T> Items => _list;`	❌	✅（通过方法）	聚合根持有子集合	外部不能改集合结构，但如果元素是 `class`，仍可能改元素本身

你想表达的约束	推荐写法（最常见）	外部能不能改	类内部能不能改	典型场景	备注/踩坑点
集合彻底隔离外部修改（防御式拷贝）	`public IReadOnlyList<T> Items => _list.ToList();`	❌	✅	高安全边界	有拷贝成本；列表很大/频繁访问要谨慎
只能通过统一入口修改集合	只暴露 `Add/Remove` 方法，不暴露 `List<T>`	❌	✅（仅通过方法）	任何需要不变量的集合	最推荐：规则检查、去重、容量限制集中在方法里
“按值比较”的不可变数据模型	`record`（默认 `record class`）	取决于设计	取决于设计	DTO、配置、返回结果、值对象	`record` 默认是引用类型，但相等性按值；常配合 `init`
高性能值对象（值类型 + 值语义）	`readonly struct` 或 `record struct`	❌（readonly）	❌（readonly）	小而稳定的值：Index/Range/Money	struct 别做太大，否则拷贝贵；尽量保持小、不可变

把表里的思路放回学生管理系统，就会发现它们经常组合出现：`Student` 这种实体更像“外部只读、内部可控改”，所以用 `class` + `private set` + 行为方法；`StudentGroup` 这种聚合根通常需要管理一个集合，但不希望外部绕过校验直接改 `List`，所以集合字段用 `readonly` 保存引用稳定性，对外只暴露 `IReadOnlyList` 视图，修改只能通过 `AddStudent/RemoveStudent`。而配置或 DTO 则很适合 `init`：初始化时组装好，之后冻结，减少随意改动导致的状态不一致。

下面的代码仍然延续 StudentManagerApp，并在 `Main` 里演示“外部读不到 set、只能通过方法改集合、init 只能在初始化阶段赋值”的感觉。

```
#nullable enable
using System;
using System.Collections.Generic;

namespace StudentManagerApp
{
    public class Student
    {
        public int Id { get; }
        public string Name { get; private set; }

        public Student(int id, string name)
        {
            ArgumentNullException.ThrowIfNull(name);
            Id = id;
            Name = name.Trim();
        }

        public void Rename(string name)
        {
            ArgumentNullException.ThrowIfNull(name);
            Name = name.Trim();
        }

        public override string ToString() => $"Student(Id={Id}, Name={Name})";
    }

    public class StudentGroup
```

```
    {
        private readonly List<Student> _students = new();

        public string GroupName { get; }
        public IReadOnlyList<Student> Students => _students;

        public StudentGroup(string groupName)
        {
            GroupName = string.IsNullOrWhiteSpace(groupName) ? "Default" : groupName.Trim();
        }

        public void AddStudent(Student s)
        {
            if (s == null) throw new ArgumentNullException(nameof(s));
            if (_students.Exists(x => x.Id == s.Id)) return;
            _students.Add(s);
        }

        public bool RemoveStudent(int studentId)
        {
            int idx = _students.FindIndex(x => x.Id == studentId);
            if (idx < 0) return false;
            _students.RemoveAt(idx);
            return true;
        }
    }

    public sealed class StudentGroupConfig
    {
        public string GroupName { get; init; } = "Default";
        public int MaxSize { get; init; } = 30;
    }

    public static class Program
    {
        public static void Main()
        {
            var g = new StudentGroup("  Class A  ");
            g.AddStudent(new Student(1, "Alice"));
            g.AddStudent(new Student(2, "Bob"));
            g.AddStudent(new Student(2, "Bob Duplicate"));

            Console.WriteLine($"Group: {g.GroupName}");
            Console.WriteLine("Students:");
            foreach (var s in g.Students)
                Console.WriteLine($"  - {s}");

            Console.WriteLine($"Remove Id=1: {g.RemoveStudent(1)}");
            Console.WriteLine($"Remove Id=99: {g.RemoveStudent(99)}");

            var cfg = new StudentGroupConfig { GroupName = "Freshmen", MaxSize = 50 };
```

```
            Console.WriteLine($"Config: GroupName={cfg.GroupName}, MaxSize={cfg.MaxSize}");
        }
    }
}
```

访问修饰符

业务代码里常用 `public/private/protected/internal` 就够了，但读框架源码时经常会遇到 `protected internal` 和 `private protected` 。它们其实是在把“继承可见性（protected）”和“程序集可见性（internal）”这两条规则做更细的组合，目的很明确：既要给继承扩展留入口，又要控制哪些入口只能在本程序集里用，避免外部用户依赖内部细节导致未来难以演进。最容易记的方式是把它们当成集合运算：`protected internal` 更像“并集”（满足任一条件就能访问），`private protected` 更像“交集”（两个条件同时满足才行）。

下面两张表把规则拆得更直观：第一张是“修饰符含义与用途”，第二张是“在不同访问场景下到底能不能访问”。

修饰符	谁能访问（直觉）	适合用在什么地方	记忆方式
`internal`	同一程序集内任何代码	模块内部实现细节、只想给同项目/同库内部使用的 API	“同项目可见”
`protected`	任何程序集内的子类（通过继承访问）	给子类扩展/覆盖实现（常配合 `virtual/override` ）	“给继承留口子”
`protected internal`	同程序集任何代码 **或** 任意程序集的子类	本程序集里到处能用，同时也允许外部通过继承扩展	“protected ∪ internal”
`private protected`	仅同程序集的子类	只给框架内部派生类型用的扩展点，不希望外部子类依赖	“protected ∩ internal”

访问者场景（你站在“调用方”的视角）	`protected internal` 能访问吗	`private protected` 能访问吗	直觉
同程序集、非子类（普通类）	✅	❌	并集满足 internal；交集缺少 protected
同程序集、子类	✅	✅	两者都满足（internal + protected）
不同程序集、子类	✅	❌	并集满足 protected；交集缺少 internal
不同程序集、非子类	❌	❌	两者都不满足

下面用学生管理系统做一个“核心管理器 + 内部派生实现”的例子。在同一个程序集里，你会看到：普通代码能调用 `protected internal` 的成员，但只有“同程序集的子类”才能调用 `private protected` 的成员。至于“不同程序集的子类是否能访问”，在一个项目里不容易直接演示出来，所以我把它的结论放在上面的场景表里（那一行最关键：外部子类能访问 `protected internal` ，但不能访问 `private protected` ）。

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public class StudentManagerCore
    {
        internal void RebuildIndexes()
        {
            Console.WriteLine("RebuildIndexes: internal (same assembly only)");
        }

        protected virtual void OnStudentChanged()
        {
            Console.WriteLine("OnStudentChanged: protected (subclass hook)");
        }

        // protected internal：同程序集任意位置可调用；外部程序集的子类也可调用
        protected internal void NotifyChanged()
        {
            Console.WriteLine("NotifyChanged: protected internal");
            OnStudentChanged();
        }

        // private protected：只能在“同程序集的子类”里访问
        private protected void InternalOnlyExtensionPoint()
        {
            Console.WriteLine("InternalOnlyExtensionPoint: private protected");
        }
    }

    // 同程序集内的派生类：两种组合修饰符都能访问
    internal class StudentManagerInternal : StudentManagerCore
    {
        public void Demo()
        {
            NotifyChanged();              // OK：protected internal
            InternalOnlyExtensionPoint(); // OK：private protected（同程序集 + 子类）
        }

        protected override void OnStudentChanged()
        {
            Console.WriteLine("Internal override: OnStudentChanged");
        }
    }

    // 同程序集内的普通类（不是子类）：只能访问 protected internal，不能访问 private protected
    internal static class StudentManagerHelper
    {
        public static void Demo(StudentManagerCore core)
        {
            core.NotifyChanged(); // OK：同程序集（internal 条件满足）
            // core.InternalOnlyExtensionPoint(); // 编译不过：不是子类（缺少 protected 条件）
```

```
        }
    }

    public static class Program
    {
        public static void Main()
        {
            Console.WriteLine("=== same assembly, non-subclass ===");
            StudentManagerHelper.Demo(new StudentManagerCore());

            Console.WriteLine("=== same assembly, subclass ===");
            new StudentManagerInternal().Demo();
        }
    }
}
```

如果你以后在框架源码里看到 `protected internal` ，通常可以把它理解成“框架愿意让外部通过继承来用，但也需要在框架内部大量调用”；而看到 `private protected` ，大多意味着“这是框架内部派生链条用的插槽，作者不想承诺给外部子类”，这样即使未来改动实现也不容易破坏第三方继承代码。

运行时补强

（这一组内容偏“读源码/写得稳”，仍然属于运行时与工程化补充。基础用法已在前面讲过，这里补上常见坑与面试追问。）

常见类形态：static、partial、嵌套类

这三种形态基本都属于“组织代码”的工具，重点是读代码时别被它们的外形吓到： `static class` 让你明确“这个类型不需要实例”； `partial class` 让你明白“同一个类型可能分散在多个文件里”；嵌套类让你知道“某个辅助类型只服务于外部类，所以被放进去了避免污染命名空间”。下面把最常遇到的规则放在一张表里，看到源码时对照即可。

形态	你会看到的写法	它解决的问题	典型场景	常见误会/注意点
静态类	`public static class X`	不让人实例化，专门放“工具函数/扩展方法/常量”	`StudentValidators` 、 `StudentFormatters` 、 `Extensions`	静态类不能被 `new` ；成员几乎都得是 `static` ；它不是“单例”，它只是“根本没有实例”
分部类	`public partial class X` （同名同命名空间）	一个类拆多个文件，便于生成代码/分模块维护	WinForms/WPF designer、Source Generator、巨大管理器拆 Core/Commands	`partial` 不是继承关系；多个文件最终会合并成一个类型；同名必须同命名空间

形态	你会看到的写法	它解决的问题	典型场景	常见误会/注意点
嵌套类	`class Outer { private class Inner { } }`	把只在某个类型内部用的辅助类型“就近放置”，减少命名污染	`CommandResult`、内部Builder、内部比较器	嵌套类仍然是类型，只是作用域更小；可见性也受 `private/internal` 控制

示例（沿用 StudentManagerApp，并在 Main 里简单演示“静态调用”“partial 合并”“嵌套类只在内部可见”这三件事）：

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public static class StudentValidators
    {
        public static bool IsValidAge(int age) => age is ≥ 0 and ≤ 150;
    }

    // 同一个 StudentManager 被拆成多个 partial 部分（通常放在不同文件）
    public partial class StudentManager
    {
        public void AddStudent(string name, int age)
        {
            if (!StudentValidators.IsValidAge(age)) throw new ArgumentOutOfRangeException(nameof(age));
            Console.WriteLine($"AddStudent: {name}, age={age}");
        }
    }

    public partial class StudentManager
    {
        private class CommandResult
        {
            public bool Ok { get; init; }
            public string Message { get; init; } = "";
            public override string ToString() => $"Ok={Ok}, Message={Message}";
        }

        // 嵌套类通常只给外部类内部使用
        private CommandResult ExecuteInternalCommand(string command)
            => new CommandResult { Ok = true, Message = $"Executed: {command}" };

        public void DemoCommand()
        {
            Console.WriteLine(ExecuteInternalCommand("rebuild-index"));
        }
```

```
    }

    public static class Program
    {
        public static void Main()
        {
            Console.WriteLine(StudentValidators.IsValidAge(18)); // 静态类：直接调用
            var mgr = new StudentManager();                      // partial：对外仍然是一个类
            mgr.AddStudent("Alice", 20);
            mgr.DemoCommand();

            // mgr.CommandResult ... // 编译不过：嵌套类是 private
        }
    }
}
```

多线程

现代 .NET 更推荐 `Task/async-await`，但 Thread 仍是读旧代码与理解底层并发的重要入口。Thread 的直觉是：把一段工作放到另一条执行路径里跑，从而避免阻塞当前流程；与之相伴的坑是：线程共享内存，因此共享数据需要同步或避免共享。这里的示例只做到“看得懂 + 会写一个最小可用的后台线程”，并采用协作式退出（用一个标志告诉线程停止，再 Join 等它优雅结束）。

方法/属性	你需要记住的点	StudentManager 场景类比
`Start()`	启动线程，只能启动一次	后台导入/导出数据
`Join()`	等线程结束（阻塞当前线程）	等导出完成再提示用户
`Sleep(ms)`	让当前线程暂停（演示/节流）	间隔保存/打印进度
`IsBackground`	后台线程不阻止进程退出	控制台 demo 常用

```
#nullable enable
using System;
using System.Threading;

namespace StudentManagerApp
{
    public partial class StudentManager
    {
        private volatile bool _stopAutosave;
        private Thread? _autosaveThread;

        public void StartAutoSave()
        {
            if (_autosaveThread != null && _autosaveThread.IsAlive) return;

            _stopAutosave = false;
            _autosaveThread = new Thread(() =>
            {
```

```
                while (!_stopAutosave)
                {
                    Console.WriteLine("[Autosave] saving ... count=" + Count);
                    Thread.Sleep(1000);
                }
                Console.WriteLine("[Autosave] stopped.");
            })
            { IsBackground = true };

            _autosaveThread.Start();
        }

        public void StopAutoSave()
        {
            _stopAutosave = true;
            _autosaveThread?.Join();
        }
    }
}
```

不安全代码

`unsafe` 允许你使用指针与直接内存操作，这会绕过一部分运行时安全保护，因此风险更高，出错更可能直接崩溃或产生不可预期行为。工程上它通常只在与原生库互操作或极限性能场景中出现；对初学者更重要的是：读到它不慌，知道 `fixed` 的作用是把托管对象地址“钉住”，避免 GC 移动导致指针失效，同时明白指针不要带出 `fixed` 作用域。

语法/关键点	含义	你需要记住的注意点
`type* p`	声明指针	指针无效会崩溃
`&x`	取地址	只能对可取地址的变量
`*p`	解引用	不做边界检查
`fixed`	固定托管对象地址	指针不要带出 fixed 块
`IntPtr` / `nint`	指针尺寸相关类型	64 位下不要随意转 int

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public static class StudentBinaryHelper
    {
        public static unsafe int ReadVersion(byte[] data)
        {
            if (data == null || data.Length < 4) throw new ArgumentException("data 长度不足4");

            fixed (byte* p = data)
```

```
            {
                int* pi = (int*)p;
                return *pi;
            }
        }
    }
}
```

C# 项目构建

创建项目与目录结构

`dotnet` 是 .NET 的命令行入口。你可以把它理解成“创建项目、构建、运行、测试、发布”的统一遥控器。最常见的两个模板是 console 和 webapi：

```
# 创建一个控制台项目
dotnet new console -n PricingApp

# 创建一个最小 Web API 项目（ASP.NET Core）
dotnet new webapi -n PricingApi
```

运行完你会得到一个目录，里面最关键的是 `*.csproj` 文件。它是“项目清单”，决定了：这个项目目标运行时是什么（net8.0 还是 net6.0）、启不启用可空、引用哪些包、编译怎么配置。你可以先把它当成 .NET 版的“项目配置入口”。

`sln`（解决方案）不是必须，但一旦项目变大就很常见。它更像“一个工作区”，把多个项目（业务、基础库、测试）组织在一起，让 IDE 和构建工具知道“这些项目是一组”。最典型的组织方式是：业务代码放 `src/`，测试放 `tests/`。

```
MyCompanyApp/
  MyCompanyApp.sln
  src/
    PricingApp/
      PricingApp.csproj
      Program.cs
    PricingApp.Core/
      PricingApp.Core.csproj
       ...
  tests/
    PricingApp.Tests/
      PricingApp.Tests.csproj
       ...
```

你可以用 CLI 生成这种结构（下面这套命令非常实用，面试也能讲）：

```
mkdir MyCompanyApp && cd MyCompanyApp
dotnet new sln -n MyCompanyApp

mkdir src tests
```

```
dotnet new classlib -n PricingApp.Core -o src/PricingApp.Core
dotnet new console  -n PricingApp      -o src/PricingApp
dotnet new xunit    -n PricingApp.Tests -o tests/PricingApp.Tests

dotnet sln add src/PricingApp.Core/PricingApp.Core.csproj
dotnet sln add src/PricingApp/PricingApp.csproj
dotnet sln add tests/PricingApp.Tests/PricingApp.Tests.csproj

# 项目引用：让 App 依赖 Core（而不是复制代码）
dotnet add src/PricingApp/PricingApp.csproj reference src/PricingApp.Core/PricingApp.Core.csproj
dotnet add tests/PricingApp.Tests/PricingApp.Tests.csproj reference src/PricingApp.Core/PricingApp.Core.csproj
```

面试里如果对方问“你怎么拆项目”，你可以用一句话回答：我会把可复用的业务规则放在 class library（Core），应用层（Console/Web）只做编排与 I/O，测试项目引用 Core 做单测与集成测。

项目配置

`.csproj` 是 XML，但你不需要背 XML。你只要认识几个最常用的开关，并能讲清它们的意义：**让编译器更早发现问题，让项目更一致、更可维护。**

一个最小的示意（不同模板可能默认值略有差异）：

```xml
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <LangVersion>latest</LangVersion>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

`TargetFramework` 决定你的程序面向哪个 .NET 运行时（比如 net8.0）。这会影响可用 API、发布方式和运行环境要求。面试里你可以说：我会跟随公司/项目的 LTS 版本，服务端一般 prefer 新 LTS。

`Nullable` 建议在新项目里开启。它能把大量 “运行时报 NRE” 的问题提前变成编译期提示。很多团队把它当成“质量开关”，越早开越省心。

`ImplicitUsings` 会自动引入常用 using，减少样板代码。团队里开启后，代码更干净，但也要注意：不同项目要统一，否则复制代码时会出现“这边能编译，那边缺 using” 的奇怪体验。

`LangVersion` 决定你能用哪些语言特性。一般不建议乱设太超前；如果你要用 record、pattern matching 之类现代语法，确保版本一致，避免“本机能编译、CI 不行”。

`TreatWarningsAsErrors` 是一个非常工程化的开关：把警告当错误，逼迫团队及时处理潜在问题。你不必一上来全开（老项目会炸），但在新项目里开启很有效，尤其配合 Nullable。

依赖管理

NuGet 是 .NET 的包管理。大多数第三方库（JSON、Redis、HTTP、日志、测试）都来自 NuGet。最常用命令是：

```
dotnet add src/PricingApp/PricingApp.csproj package Dapper
dotnet add src/PricingApp/PricingApp.csproj package Microsoft.Extensions.Http
```

面试里常见追问是“版本冲突怎么处理”。你不需要背一堆细节，能讲清三点就很够用：

第一，NuGet 有传递依赖：你引用 A，A 又引用 B。版本冲突通常来自“不同包要求不同版本的 B”。
第二，最常见处理方式是“显式引用对齐版本”：你在项目里直接引用 B 的某个版本，让依赖树收敛到一致。
第三，尽量用集中管理版本（比如 Directory.Packages.props）或至少团队约定统一版本，避免各项目各配各的。

构建、运行、测试、发布

这四条是你在任何 .NET 项目里都会用的基本功：

```
dotnet build     # 编译（检查语法、生成中间产物）
dotnet run       # 运行（会隐式 build）
dotnet test      # 运行测试（会隐式 build）
dotnet publish   # 发布（生成可部署产物）
```

`build` 是“把代码编译出来”，你可以把它看成最基础的静态检查。
`run` 是“编译后启动”，开发阶段常用，它会先 build 再跑。
`test` 会构建并执行测试，是 CI 里最常见的一步。
`publish` 是交付关键：它会生成发布目录，包含运行所需文件。面试里经常问“build 和 publish 什么区别”，你可以说：build 产物偏开发/编译检查；publish 产物偏交付/部署，会按目标环境输出可运行包。

最小可运行 Web API

如果你面试的是后端岗位，Minimal API 是非常好的“最小示例”，因为它短，但能把 DI、配置、日志三件套一次讲清。

下面这段就是 `dotnet new webapi` 模板的典型风格（做了轻微简化）：

```
using Microsoft.AspNetCore.Builder;
using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Microsoft.Extensions.Logging;

var builder = WebApplication.CreateBuilder(args);

// 配置：来自 appsettings.json / 环境变量 / 命令行等
// 这里示意读取一个配置项
var greeting = builder.Configuration["Greeting"] ?? "Hello";

// DI：注册服务（这里用一个很小的服务示例）
builder.Services.AddSingleton<Clock>();

var app = builder.Build();

```

```
// 日志：ILogger 来自 DI
app.MapGet("/time", (Clock clock, ILoggerFactory loggerFactory) =>
{
    var logger = loggerFactory.CreateLogger("TimeEndpoint");
    logger.LogInformation("Time endpoint called");
    return new { greeting, now = clock.NowIso() };
});

app.Run();

public sealed class Clock
{
    public string NowIso() => DateTimeOffset.UtcNow.ToString("O");
}
```

你可以这样解释“三件套”各自解决什么问题：DI 负责组装与可替换性（测试也更容易）；配置负责多环境差异（开发/预发/生产）；日志负责可观测性（线上排障靠它）。面试官如果继续问“生命周期怎么选”，你就从 Singleton/Scoped/Transient 的直觉讲起。

发布与交付

发布时有两种常见方式。你不需要背很多参数，但要能讲清选择依据：目标机器有没有 .NET 运行时。

Framework-dependent：发布包更小，但运行环境必须预装对应 .NET 运行时。适合公司有统一运行时基建的场景。

Self-contained：把运行时也一起打包，发布包更大，但目标机器不需要预装 .NET。适合交付到不受控环境、或你希望减少部署依赖的场景。

你可以这样发布（示意）：

```
# Framework-dependent（默认就是这类）
dotnet publish -c Release -o out

# Self-contained（指定运行时标识 RID，例如 linux-x64）
dotnet publish -c Release -r linux-x64 --self-contained true -o out
```

面试里你补一句“发布后我会用实际环境验证启动、健康检查与关键路径”，就非常工程：你不是只会出包，你会交付。

进阶工具链

上面那一套足够你把项目跑起来，但真正进公司做项目，你会很快遇到一些“不是语法问题、却天天卡你”的工程细节。这里我把它们分成两层：**必会**（你很快就会用到）和 **团队级进阶**（多项目/多人协作时更常见）。为了不显得臃肿，这一段我把相近主题合并在一起：读的时候按“问题类型”记，比按小标题记更轻。

个人级

依赖还原与版本稳定
很多人只会 `dotnet build`，但在 CI 里经常会拆成 `restore` + `build`：先还原依赖，再编译。原因很简单：依赖还原失败和代码编译失败是两类问题，分开更容易定位。

```
dotnet restore
dotnet build
```

如果团队特别在意“每次还原出来的依赖版本必须一致”，可以启用锁文件 `packages.lock.json`。它的直觉是：把依赖树的最终解析结果锁住，避免某天 NuGet 源更新导致依赖树微妙变化，出现“昨天还好好的今天就坏了”。

环境、配置与本地调试

ASP.NET Core 默认有环境概念。开发机常见是 `Development`，线上是 `Production`。你可以通过环境变量控制：

- `ASPNETCORE_ENVIRONMENT=Development|Staging|Production`

配置的来源通常会叠加：`appsettings.json` → `appsettings.{Environment}.json` → 环境变量 → 命令行参数。最后谁覆盖谁，按优先级决定。这件事在排障时很关键：你以为配了，实际上被环境变量覆盖了。

开发阶段还有一个很常用的工具：User Secrets（用户机密）。它把敏感信息存在本机而不是提交到仓库，适合本地开发的连接串、API key。

```
dotnet user-secrets init
dotnet user-secrets set "ConnectionStrings:Main" "Server=...."
```

如果你用 VS/VS Code 或 `dotnet run` 调试 web 项目，经常会看到 `Properties/launchSettings.json`。它主要解决：本地启动用哪个端口、是否打开浏览器、注入哪些环境变量。很多“我本地为什么是 5001 端口”的问题都在这里。

对外调用与日志（线上排障靠这两样）

在服务端项目里，对外调用几乎不可避免。很多人会手写 `new HttpClient()`，短期看没问题，长期可能遇到连接数耗尽、DNS 变更不生效等问题。ASP.NET Core 推荐用 `IHttpClientFactory` 管理连接与生命周期。

```
builder.Services.AddHttpClient("github", c =>
{
    c.BaseAddress = new Uri("https://api.github.com/");
    c.Timeout = TimeSpan.FromSeconds(3);
});
```

面试里你可以说：我会用 HttpClientFactory 配合 Polly 做超时/重试/熔断，避免手写连接管理。

`ILogger` 是抽象，它需要 provider 把日志输出到控制台、文件、或日志系统。开发阶段最常见是 Console；生产更常见是接 Serilog/ELK/云厂商日志。面试里别说“我会打日志”，要说“我会打结构化日志”：不要拼字符串，尽量用模板和字段，方便检索聚合。

```
logger.LogInformation("Pay finished. orderId={OrderId} cost={Cost}", orderId, cost);
```

测试组织（让改动可控）

你已经会 `dotnet test`，但工程里更常见的组织是：单测项目引用 Core，不依赖外部系统；少量集成测试用 `WebApplicationFactory` 启动内存版服务，验证关键 API 链路。这样你的测试既快又可信。

```
dotnet test --filter Category=Unit
dotnet test --filter Category=Integration
```

团队级

固定 SDK 与环境一致性
你可能见过这样的尴尬：你本机装了 .NET 8，能编译；同事机器还在 .NET 7，CI 用的也是 7，于是 PR 一合就红。`global.json` 的作用就是把“我们用哪个 SDK”写死在仓库里，让所有人（包括 CI）用同一套工具链。

```
{
  "sdk": {
    "version": "8.0.200",
    "rollForward": "latestPatch"
  }
}
```

面试里你可以这么说：我会在仓库根目录放 `global.json` 固定 SDK，避免环境漂移；如果公司有统一版本策略，就按公司版本走。

集中管理依赖版本
当你的解决方案里有多个项目，每个项目都写一遍 `<PackageReference Version="...">` 会非常痛苦：升级要改很多地方，也容易出现版本不一致。集中管理的思路是把版本放到仓库根目录统一维护，各项目只写引用不写版本。

```
<!— Directory.Packages.props —>
<Project>
  <ItemGroup>
    <PackageVersion Include="Dapper" Version="2.1.35" />
    <PackageVersion Include="Serilog.AspNetCore" Version="8.0.1" />
  </ItemGroup>
</Project>
```

然后项目里只写：

```
<ItemGroup>
  <PackageReference Include="Dapper" />
</ItemGroup>
```

你在面试里讲出这一点，会让人觉得你写过多项目工程，而不是只写过单个 demo。

统一工程配置与规则
当项目变大，你会想要“所有项目都开启 Nullable”“警告当错误”“统一语言版本”。把它们逐个写进每个 csproj 很容易遗漏。更常见的做法是用 `Directory.Build.props` 统一下发。

```
<!— Directory.Build.props —>
<Project>
  <PropertyGroup>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
  </PropertyGroup>
</Project>
```

再配合 `.editorconfig`（风格）和 Analyzer（规则），你就能把很多争论从“人吵架”变成“工具统一裁决”。这也是“工程质量”最省心的一条路。

发布选项（了解即可）
你不需要在笔记里把 publish 参数写成百科，但国内面试偶尔会问“你们怎么发布的”，你至少要知道这些词是什么意思：

- `-c Release`：发布用 Release 配置（优化与裁剪策略不同）
- `-r linux-x64`：指定目标运行时（RID）
- `--self-contained true/false`：是否自带运行时
- 单文件发布、ReadyToRun、Trimming：减小体积、提升冷启动、减少依赖（但 trimming 可能删掉反射用到的类型，需要谨慎）
- NativeAOT：更激进的 AOT，换取更快启动/更小体积，但限制更多（了解即可）

你在面试里用一句话收尾就很好：我知道 publish 的选择取决于部署环境与体积/启动/兼容性的权衡，项目越大越需要用数据（镜像大小、启动时间、内存）来验证选择是否正确。

你把这些坑位讲出来，面试官通常会判断你真正做过交付，而不是只写过 demo。

进阶与查漏补缺

record、required 与现代语法

如果你只用“C# 语法基础”写业务，其实能做的事已经很多，但国内面试经常会在中后段突然问你一句：“你平时用哪些现代 C# 特性？为什么用？”这类问题不是考你背版本号，而是在看你能不能用语言特性把代码写得更安全、更好读、更少 Bug。下面这些点，不用死记时间线，只要能讲清楚“它解决了什么麻烦”。

最常见的一类是“表达数据结构”。以前我们写 DTO/参数对象，常常要手写一堆样板代码：构造函数、Equals、GetHashCode、ToString。`record` 让你更自然地表达“数据承载 + 值相等”，而且配合 `with` 做复制修改非常舒服。如果你希望它仍然是值语义但又要性能，可以考虑 `record struct`。面试里最稳的说法是：当对象主要用来承载数据、比较内容是否相等时，用 record 可以减少样板代码并降低出错概率；当对象有明显生命周期与身份时（比如订单），还是用 class + 领域行为更合适。

第二类是“更强的表达能力”。`switch` 表达式和模式匹配（pattern matching）让你从一坨 if-else 里解放出来，尤其适合写状态机、类型分发和解析逻辑。你不需要为了炫技而用它，但当分支变多时，它会让逻辑更集中、更不容易漏分支。面试官常追问的点是：模式匹配不仅能 match 值，还能 match 类型、属性结构，并且支持 `when` 守卫。

第三类是“把错误更早暴露”。你已经写了可空引用（Nullable Reference Types），这里再补一个经常配套出现的关键词：`required`。它表达“这个属性在对象初始化时必须被赋值”，能把很多运行时 NRE 变成编译期提示。面试里你可以把它讲成：我更希望错误发生在编译期，而不是线上。

下面给一个短例子，把 `record`、`with`、`switch` 表达式和模式匹配串起来看，感受一下它的“表达力”。

```
using System;

public record Address(string City, string Detail);

public abstract record PaymentResult
{
    public sealed record Success(string TxnId) : PaymentResult;
```

```
    public sealed record Failed(string Reason) : PaymentResult;
}

public static string ToMessage(PaymentResult r) => r switch
{
    PaymentResult.Success { TxnId: var id } => $"支付成功：{id}",
    PaymentResult.Failed { Reason: var reason } => $"支付失败：{reason}",
    _ => "未知结果"
};

var addr = new Address("SH", "Road 1");
var updated = addr with { Detail = "Road 2" };
```

异常体系进阶

异常在 C# 里属于“语言级的错误传播机制”，不是日志、不是返回码的替代品。工程里最常见的坑不是“不会 try-catch”，而是两类：一种是把可预期分支全用异常表达，导致代码难读、性能差；另一种是 catch 之后“换个异常重新抛”，把原始堆栈信息弄丢，最后排查时只看到你自己这层。

这里把你需要记住的点整理成一张表：

你在代码里看到的写法	直觉含义	面试/排查时的关键点
`throw;`	原样重新抛出当前异常	✅ 保留原始堆栈（stack trace），排查最重要
`throw ex;`	把捕获到的异常当“新异常”抛出	❌ 会重置堆栈，等于抹掉真实出错位置（面试常问）
`throw new XException("msg", ex);`	包一层更贴近语义的异常	✅ 用 `InnerException` 保留根因；msg 提供业务上下文
`catch(Exception) { ... }`	把所有异常吞掉/统一处理	只有在边界层（Host/UI）才这么干；中间层尽量具体捕获
`finally { ... }`	无论是否异常都执行	常用于“必须执行的清理/回滚”；但更推荐 `using` 处理 IDisposable

StudentManager 里最常见的异常分层直觉是：入参错误（例如空 id）用 `ArgumentException`；违反约束（重复 id）用 `InvalidOperationException`；底层 IO 失败（文件读写）让 `IOException` 自然冒泡到边界层统一处理，并且在边界层记录日志。

```
using System;

namespace StudentManagerApp
{
    public sealed class StudentStoreException : Exception
    {
        public StudentStoreException(string message, Exception inner) : base(message, inner) { }
    }

```

```
10      public static class Demo_Exception
11      {
12          static void DoIO()
13          {
14              try
15              {
16                  // 这里用一个必定失败的路径演示
17                  System.IO.File.ReadAllText("/path/not-exist/students.csv");
18              }
19              catch (System.IO.IOException ex)
20              {
21                  // ✅ 包一层业务语义异常，并保留 InnerException
22                  throw new StudentStoreException("读取学生数据失败（IO 层异常）", ex);
23              }
24          }
25
26          public static void Main()
27          {
28              try
29              {
30                  DoIO();
31              }
32              catch (StudentStoreException ex)
33              {
34                  Console.WriteLine(ex.Message);
35                  Console.WriteLine("Root ⇒ " + ex.InnerException?.GetType().Name);
36              }
37
38              try
39              {
40                  try
41                  {
42                      throw new InvalidOperationException("boom");
43                  }
44                  catch (Exception ex)
45                  {
46                      // ❌ 演示：throw ex 会重置堆栈（不要这么写）
47                      // throw ex;
48
49                      // ✅ 演示：throw; 保留原始堆栈
50                      throw;
51                  }
52              }
53              catch (Exception ex)
54              {
55                  Console.WriteLine("rethrow ⇒ " + ex.GetType().Name);
56              }
57          }
58      }
59  }
```

async/await 进阶

超时/取消与并发控制

你已经提醒了不要 `.Result/.Wait()` ，这是对的。国内面试更进一步常问三件事：取消、并发控制、以及“什么时候用 ValueTask”。取消的关键不在于“传一个 CancellationToken”，而在于你能把 token 贯穿到下游 I/O，并且在超时/用户取消时及时停止工作。一个很常见的工程化写法是“超时包装”：用 `CancellationTokenSource` 合并外部 token 与超时 token。

并发控制则常问：你如何限制同时跑的任务数量？最常见的答案是 `SemaphoreSlim` ，它比 `lock` 更适合控制异步并发。至于 `ValueTask` ，面试里不需要深入实现，只要知道：当一个异步方法“多数情况下同步完成”并且你想减少分配时才考虑它；否则 `Task` 更简单可靠。

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;

public static async Task<T> WithTimeout<T>(Task<T> task, TimeSpan timeout,
CancellationToken ct)
{
    using var cts = CancellationTokenSource.CreateLinkedTokenSource(ct);
    cts.CancelAfter(timeout);
    return await task.WaitAsync(cts.Token);
}

public static async Task RunLimitedAsync(IEnumerable<Func<Task>> jobs, int parallelism)
{
    using var sem = new SemaphoreSlim(parallelism);
    var tasks = jobs.Select(async job =>
    {
        await sem.WaitAsync();
        try { await job(); }
        finally { sem.Release(); }
    });
    await Task.WhenAll(tasks);
}
```

你写业务代码时，大多数 async/await 的正确用法是“不要阻塞、一路 await”。面试常追问的点主要集中在三个方向：第一是 `Task` 与线程的关系（Task 不等于 Thread，它是异步操作的抽象）；第二是取消（ `CancellationToken` ）怎么贯穿调用链；第三是为什么有些库代码会写 `ConfigureAwait(false)` 。

关键点	你需要形成的直觉	代码里的典型信号
`async` 方法返回 `Task/Task<T>`	异步操作的“句柄”；可以 await、可以组合	`await SomethingAsync()`
不要用 `.Result/.Wait()`	在有同步上下文（UI/ASP.NET）时可能死锁；也会阻塞线程	面试常问“为什么死锁”

关键点	你需要形成的直觉	代码里的典型信号
`CancellationToken` 需要一路传下去	取消是“协作式”的：你请求取消，对方检查 token 并退出	`ThrowIfCancellationRequested()`
`ConfigureAwait(false)`	库代码通常不需要回到原上下文；避免不必要的上下文切换	出现在 SDK/底层库里更常见

下面用 StudentManager 的“导出学生列表到文件”演示：支持取消、支持进度打印，并且展示 `await using`（异步释放）这一类现代写法。

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace StudentManagerApp
{
    public record Student(int Id, string Name, int Age);

    public static class StudentExporter
    {
        public static async Task ExportAsync(
            IEnumerable<Student> students,
            string path,
            CancellationToken ct)
        {
            ct.ThrowIfCancellationRequested();

            await using var fs = new FileStream(path, FileMode.Create, FileAccess.Write, FileShare.Read);
            await using var sw = new StreamWriter(fs, Encoding.UTF8);

            int i = 0;
            foreach (var s in students)
            {
                ct.ThrowIfCancellationRequested();
                await sw.WriteLineAsync($"{s.Id},{s.Name},{s.Age}");

                if ((++i % 2) == 0)
                    Console.WriteLine($"exported {i} rows ... ");

                // 模拟一点耗时
                await Task.Delay(200, ct);
            }
        }
    }

    internal static class Demo_Async
    {
```

```
        public static async Task Main()
        {
            var students = new List<Student>
            {
                new(1, "Alice", 20),
                new(2, "Bob", 18),
                new(3, "Cathy", 22),
                new(4, "Dan", 19),
            };

            using var cts = new CancellationTokenSource();
            cts.CancelAfter(650); // 演示：过一会儿取消

            try
            {
                await StudentExporter.ExportAsync(students, "students.csv", cts.Token);
                Console.WriteLine("export done");
            }
            catch (OperationCanceledException)
            {
                Console.WriteLine("export canceled");
            }
        }
    }
}
```

装箱/拆箱、default、可空值类型：值类型的“隐形成本”

这部分是国内面试里非常高频的“基础但容易踩坑”内容。关键不是背定义，而是你能在代码里看出哪里发生了装箱，哪里会产生额外分配。

现象	根因	你在代码里怎么避免
`object o = 123;`	`int` 是值类型，赋给 `object` 会装箱（分配对象）	用泛型 `T` 或保持值类型语义
`Console.WriteLine((object)someStruct);`	struct 被当成 object 传递时装箱	尽量用泛型/重载避免走 object
`Nullable<int>` 与 `int?`	这是“可空值类型”，底层是 struct 包一层 HasValue/Value	对外表达“可能缺省”的数字
`default(T)`	泛型里拿到类型的默认值	值类型默认是 0；引用类型默认是 null

```
using System;

namespace StudentManagerApp
{
    internal static class Demo_Boxing
```

```
    {
        public static void Main()
        {
            int x = 42;
            object o = x;             // 装箱：分配一个对象
            int y = (int)o;           // 拆箱：从对象里取回 int

            Console.WriteLine($"x={x}, y={y}");

            int? score = null;
            Console.WriteLine(score.HasValue ? score.Value.ToString() : "<null>");

            Console.WriteLine(DefaultOf<int>());     // 0
            Console.WriteLine(DefaultOf<string>());  // <null>
        }

        static string DefaultOf<T>()
        {
            var v = default(T);
            return v?.ToString() ?? "<null>";
        }
    }
}
```

集合与相等性

如果一个对象要做 Dictionary 的 key 或 HashSet 的元素，你必须保证 `Equals` 与 `GetHashCode` 对同一组字段保持一致。很多隐蔽 bug 是：你重写了 Equals 但忘了重写 GetHashCode，或者你把可变字段纳入了哈希计算，然后对象进了 HashSet 之后字段变了，哈希桶就对不上了。

写法/选择	适合什么	你要特别记住的风险
实体（有 Id 的 Student）按 Id 相等	“身份相等”	Id 必须稳定；不要把 Name 这类会改的字段纳入哈希
值对象用 record	“字段值相等”	record 默认按值比较，写起来省样板
自定义比较器 `IEqualityComparer<T>`	不想污染类型本身语义	Dictionary/HashSet 构造时传 comparer

```
using System;
using System.Collections.Generic;

namespace StudentManagerApp
{
    public sealed class Student : IEquatable<Student>
    {
        public int Id { get; }
        public string Name { get; private set; }

        public Student(int id, string name)
        {
```

```
            Id = id;
            Name = name;
        }

        public void Rename(string name) => Name = name;

        public bool Equals(Student? other) => other is not null && other.Id == Id;
        public override bool Equals(object? obj) => obj is Student s && Equals(s);
        public override int GetHashCode() => Id.GetHashCode();

        public override string ToString() => $"#{Id} {Name}";
    }

    internal static class Demo_Hash
    {
        public static void Main()
        {
            var set = new HashSet<Student>();

            var a = new Student(1, "Alice");
            set.Add(a);

            Console.WriteLine(set.Contains(new Student(1, "X"))); // True：按 Id

            a.Rename("Alice2");
            Console.WriteLine(set.Contains(a)); // True：哈希基于 Id，不受 Name 变化影响
        }
    }
}
```

事件进阶——多播与订阅管理

你在代码里经常看到一个现象：同样是委托类型，普通委托字段（例如 `Action` ）在外部可以直接调用，但声明成 `event` 之后，外部只能 `+=` / `-=` ，却不能 `=` 覆盖，也不能 `Invoke` 。这不是“约定”，而是语言层面的限制： `event` 把“触发权”锁在发布者内部，外部只能管理订阅关系。工程里这条限制非常重要，因为它防止订阅者把你的通知通道当成“可写的公共字段”，避免出现谁都能清空订阅链、谁都能乱触发的情况。

当你想把订阅过程做得更受控（例如记录订阅次数、只允许某类订阅、或者把订阅者转发到另一个内部事件），C# 允许你给事件写 **自定义访问器**： `add` / `remove` 。它们看起来像属性的 get/set，但语义是“订阅/退订”。最常见的落地点并不是让你写复杂逻辑，而是让你能在这里做线程安全（加锁）或做审计记录。

写法	外部能做什么	发布者内部能做什么	典型用途
`public Action? Something;`	可以 `=` 覆盖、可以 `Invoke()`	同上	很少对外暴露；更像回调字段
`public event Action? Something;`	只能 `+=` / `-=`	可以 `Invoke()`	发布-订阅，限制触发权
`public event Action Something { add { ... } remove { ... } }`	还是只能 `+=` / `-=`	你完全接管订阅逻辑	线程安全、转发、统计、过滤

```
#nullable enable
using System;

namespace StudentManagerApp
{
    public sealed class StudentManager
    {
        // 内部真正存委托链的字段（外部拿不到）
        private EventHandler<StudentChangedEventArgs>? _studentChanged;

        // 对外暴露 event，但订阅/退订走自定义 add/remove
        public event EventHandler<StudentChangedEventArgs> StudentChanged
        {
            add
            {
                // 这里可以加锁/记录日志/限制订阅者等；演示先做简单打印
                Console.WriteLine("[Subscribe] +1");
                _studentChanged += value;
            }
            remove
            {
                Console.WriteLine("[Unsubscribe] -1");
                _studentChanged -= value;
            }
        }

        private void OnStudentChanged(Student s, string action)
            => _studentChanged?.Invoke(this, new StudentChangedEventArgs(s, action));

        public void DemoFire()
        {
            OnStudentChanged(new Student { Id = "S001", Name = "Alice", Age = 20 }, "Add");
        }
    }

    public sealed class StudentChangedEventArgs : EventArgs
    {
        public Student Student { get; }
        public string Action { get; }
        public StudentChangedEventArgs(Student student, string action) { Student = student; Action = action; }
    }

    public sealed class Student
    {
        public string Id { get; init; } = "";
        public string Name { get; init; } = "";
        public int Age { get; init; }
        public override string ToString() => $"{Id} {Name} Age={Age}";
    }

    public static class Program
```

```
    {
        public static void Main()
        {
            var mgr = new StudentManager();
            EventHandler<StudentChangedEventArgs> h = (_, e) => Console.WriteLine($"[Event] {e.Action}: {e.Student}");

            mgr.StudentChanged += h;
            mgr.DemoFire();
            mgr.StudentChanged -= h;
        }
    }
}
```

泛型进阶——约束、默认值与实战套路

在真实工程里，泛型不只是“把 T 填进去就完了”。一旦你在泛型代码里想做更具体的事情，例如你希望 T 一定有无参构造函数可以 `new T()`，或者你希望 T 一定实现了某个接口（这样你才能在泛型里调用接口方法），你就会用到 **泛型约束（generic constraints）**。约束的价值是把“运行时才会炸”的不确定性，提前变成“编译期就不让你写错”。这点在面试里也经常被问到：为什么很多框架 API 里会写 `where T : class`、`where T : new()`，它们到底限制了什么。

你想在泛型里做什么	你需要的约束	直觉含义	StudentManager 场景（例子）
想写 `new T()` 创建对象	`where T : new()`	T 必须有 public 无参构造	导入 CSV 时，先 new 一个空对象再填属性
想把“允许 null”说清楚	`where T : class` / `where T : struct`	限定引用类型或值类型	仓储返回 `T?` 时，引用/值的处理不一样
想调用某个能力（方法/属性）	`where T : ISomething`	T 必须实现接口	让 `Student` / `Teacher` 都实现 `IHasId`，仓储统一按 Id 索引
想表达“必须继承某个基类”	`where T : SomeBase`	T 必须从该基类派生	需要复用基类的默认行为/字段
想限制为“可比较”	`where T : IComparable<T>`	能参与排序比较	通用排序工具里可以直接 `CompareTo`

```
#nullable enable
using System;
using System.Collections.Generic;

namespace StudentManagerApp
{
    // 把“有 Id”抽成接口：这是一种典型的“技术层抽象”，不是理念章节
    public interface IHasId<TKey>
```

```
    {
        TKey Id { get; }
    }

    public sealed class Student : IHasId<string>
    {
        public string Id { get; init; } = "";
        public string Name { get; set; } = "";
        public override string ToString() => $"{Id} {Name}";
    }

    // 约束：T 必须是引用类型，并且实现 IHasId<TKey>
    public sealed class Repository<TKey, T> where T : class, IHasId<TKey>
    {
        private readonly Dictionary<TKey, T> _map = new();

        public bool TryGet(TKey id, out T? value) => _map.TryGetValue(id, out value);

        public void Add(T item)
        {
            if (_map.ContainsKey(item.Id))
                throw new InvalidOperationException("Duplicate id.");
            _map.Add(item.Id, item);
        }
    }

    // 约束：T 必须有 public 无参构造函数，这样才能 new T()
    public static class SimpleFactory
    {
        public static T CreateEmpty<T>() where T : new() => new T();
    }

    public static class Program
    {
        public static void Main()
        {
            var repo = new Repository<string, Student>();
            repo.Add(new Student { Id = "S001", Name = "Alice" });

            Console.WriteLine(repo.TryGet("S001", out var s) ? s!.ToString() : "<not found>");

            // new() 约束演示：Student 有无参构造（init 属性默认值也算）
            var empty = SimpleFactory.CreateEmpty<Student>();
            Console.WriteLine($"empty student: Id='{empty.Id}', Name='{empty.Name}'");
        }
    }
}
```

泛型进阶——协变与逆变

补充：协变/逆变与约束的口述模板

很多人面试被问泛型时，只能讲“类型安全、减少装箱”，这不够。国内面试更爱追问协变/逆变：为什么 `IEnumerable<Dog>` 能当成 `IEnumerable<Animal>`，但 `List<Dog>` 不行？你只要抓住一个直觉：如果一个接口“只产出 T”（只返回 T，不接收 T），那它更容易安全地向上转型，所以叫协变（`out T`）；如果“只消费 T”（只接收 T，不返回 T），那它更适合向下兼容，所以叫逆变（`in T`）。一旦既产出又消费，就很难保证类型安全，通常就不能变。

工程里最常见的例子是：`IEnumerable<out T>` 协变，`IComparer<in T>` 逆变。你在面试里能把这两个例子讲顺，就已经很加分。然后再补一句：类（比如 `List<T>`）往往不能协变/逆变，因为它既能读也能写，写入会破坏安全性。

约束（`where T : ...`）也是高频。它的核心不是语法，而是让编译器知道“你能对 T 做什么”。例如 `where T : class` 允许你写 `T?` 并用 null；`where T : new()` 允许你 `new T()`；`where T : notnull` 避免 key 为空；这些都能让错误更早暴露。

```
using System;
using System.Collections.Generic;

public static void PrintAll<T>(IEnumerable<T> items)
{
    foreach (var i in items) Console.WriteLine(i);
}

IEnumerable<string> s = new List<string>();
IEnumerable<object> o = s; // 协变：安全，因为只枚举产出

public static T Create<T>() where T : new() => new T();
```

你在面试或读源码时经常会看到 `IEnumerable<out T>` 这种写法，`out` / `in` 在这里不是 `out` 参数，而是**泛型变体（variance）**：用来描述“当类型存在继承关系时，泛型实例能不能跟着一起替换”。最直觉的例子是：`Student : Person`，那 `IEnumerable<Student>` 能不能当作 `IEnumerable<Person>` 用？在 .NET 里答案是可以的，因为 `IEnumerable<out T>` 对 T 做了协变声明，表示“只产出（返回）T，不消耗（接收）T”，因此这种替换是类型安全的。

变体	关键字	直觉	典型接口	StudentManager 里的感受
协变	`out T`	只“产出”T（返回 T），就能把更具体的当更抽象的	`IEnumerable<out T>`	把 `List<Student>` 当成“可枚举的 Person”去遍历
逆变	`in T`	只“消耗”T（接收 T），就能把更抽象的当更具体的	`IComparer<in T>`、`IEqualityComparer<in T>`	一个 `IComparer<Person>` 可以拿来比较 Student（因为比较器只接收参数）

变体	关键字	直觉	典型接口	StudentManager 里的感受
不变	无	既产出又消耗，通常就不允许替换	`List<T>`、`Dictionary<TKey,TValue>`	`List<Student>` 不能当作 `List<Person>`（否则可能把 Teacher 塞进去）

```
#nullable enable
using System;
using System.Collections.Generic;

namespace StudentManagerApp
{
    public class Person
    {
        public string Name { get; init; } = "";
        public override string ToString() => Name;
    }

    public sealed class Student : Person
    {
        public string Id { get; init; } = "";
        public override string ToString() => $"{Id} {Name}";
    }

    public static class Program
    {
        public static void Main()
        {
            // 协变：IEnumerable<out T> 允许把 IEnumerable<Student> 赋给 IEnumerable<Person>
            IEnumerable<Student> students = new List<Student>
            {
                new Student { Id="S001", Name="Alice" },
                new Student { Id="S002", Name="Bob" },
            };
            IEnumerable<Person> people = students;

            foreach (var p in people)
                Console.WriteLine(p);

            // 逆变：IComparer<in T> 允许把 IComparer<Person> 当 IComparer<Student> 用
            IComparer<Person> byName = Comparer<Person>.Create((a, b)
                => StringComparer.OrdinalIgnoreCase.Compare(a.Name, b.Name));

            // 这里可以直接用在 Student 上，因为 IComparer<in T> 是逆变
            var list = new List<Student>((IEnumerable<Student>)students);
            list.Sort(byName);

            Console.WriteLine("sorted:");
            foreach (var s in list)
                Console.WriteLine(s);
        }
```

```
    }
}
```

模式匹配与 switch 表达式

你在 C# 里做“按类型/按形状分支”时，不一定要写一长串 `if (x is ...)` 。模式匹配把“判断 + 取值”合并成一种更紧凑的写法，尤其是 `switch` 表达式，读起来像“把输入映射成输出”。它属于语言层面的多态分支工具：接口/继承解决“怎么替换实现”，模式匹配解决“我手里拿到的是基类/接口引用时，怎么安全地按具体情况处理”。

写法	典型形态	你会在项目里看到的用法
类型模式	`if (p is Student s)`	安全下转 + 立即使用 `s`
属性模式	`if (s is { Age: ≥ 18 })`	直接按属性条件分支
`switch` 表达式	`x switch { ... }`	映射、分类、格式化输出
`when` 守卫	`case ... when cond:`	复杂条件时保持分支整洁

```
using System;

namespace StudentManagerApp
{
    public abstract class Person { public string Name { get; init; } = ""; }
    public sealed class Student : Person { public int Age { get; init; } }
    public sealed class Teacher : Person { public string Subject { get; init; } = ""; }

    public static class PersonFormatter
    {
        public static string Describe(Person p) =>
            p switch
            {
                Student { Age: ≥ 18 } s => $"Adult Student: {s.Name}, Age={s.Age}",
                Student s               => $"Minor Student: {s.Name}, Age={s.Age}",
                Teacher t               => $"Teacher: {t.Name}, Subject={t.Subject}",
                _                       => p.ToString() ?? "<null>"
            };
    }

    internal static class Demo
    {
        static void Main()
        {
            Person p1 = new Student { Name = "Alice", Age = 20 };
            Person p2 = new Student { Name = "Bob", Age = 16 };
            Person p3 = new Teacher { Name = "Ms.Li", Subject = "Math" };

            Console.WriteLine(PersonFormatter.Describe(p1));
            Console.WriteLine(PersonFormatter.Describe(p2));
```

```
            Console.WriteLine(PersonFormatter.Describe(p3));
        }
    }
}
```

迭代器与 yield

`IEnumerable<T>` 表示“我能被 foreach 遍历”，但它不要求你一次性把所有元素都算出来。`yield return` 让你用最小代码写出“按需生成”的迭代器：调用方 foreach 时，才一边遍历一边产出数据。读源码时，看到 `yield` 就要立刻想到两点：它是状态机；它是惰性执行（可能多次枚举会重复执行）。

你想要的效果	更常见的实现方式
不想一次性分配大列表	`yield return` 按需产出
想把过滤逻辑写成“可枚举序列”	`IEnumerable<T>` + `yield`
想控制枚举生命周期（打开文件逐行读）	`using` + `yield` （注意资源释放）

```
using System;
using System.Collections.Generic;

namespace StudentManagerApp
{
    public sealed class Student
    {
        public string Id { get; init; } = "";
        public string Name { get; init; } = "";
        public int Age { get; init; }
        public override string ToString() => $"{Id} {Name} Age={Age}";
    }

    public sealed class StudentManager
    {
        private readonly List<Student> _list = new();

        public void Add(Student s) => _list.Add(s);

        // 惰性过滤：调用方枚举时才执行筛选
        public IEnumerable<Student> WhereAgeAtLeast(int minAge)
        {
            foreach (var s in _list)
                if (s.Age ≥ minAge)
                    yield return s;
        }
    }

    internal static class Demo
    {
        static void Main()
        {
            var mgr = new StudentManager();
```

```
            mgr.Add(new Student { Id = "S001", Name = "Alice", Age = 20 });
            mgr.Add(new Student { Id = "S002", Name = "Bob", Age = 16 });

            foreach (var s in mgr.WhereAgeAtLeast(18))
                Console.WriteLine(s);
        }
    }
}
```

LINQ 进阶——IEnumerable vs IQueryable 与表达式树

补充：多次枚举与 IQueryable 的两个坑

你已经写了延迟执行，这很好。但国内面试（尤其涉及 EF Core）会进一步追问两个坑：第一，延迟执行意味着“你枚举一次就执行一次”，所以如果你多次枚举同一个查询，它可能多次访问数据库或远程服务，造成性能问题。工程里最常见的修复是：如果你确实需要复用结果，及时 `ToList()` 或缓存结果，避免重复执行。

第二个坑是 `IEnumerable` vs `IQueryable`。它们表面上都能写 `Where/Select`，但语义不同：`IEnumerable` 处理的是内存里的集合，传递的是委托（lambda 会直接在本进程执行）；`IQueryable` 传递的是表达式树，通常会被 ORM 翻译成 SQL，在数据库侧执行。面试里最稳的表达是：我知道哪些操作会被翻译成 SQL，哪些会在客户端执行；如果不小心把不可翻译的操作放进去，可能导致全表拉回本地再过滤。

```
using System;
using System.Collections.Generic;
using System.Linq;

// IEnumerable：在内存中过滤
IEnumerable<int> xs = new[] { 1, 2, 3, 4 };
var ys = xs.Where(x => x % 2 == 0).ToList();

// IQueryable：表达式树，可能被翻译为 SQL
// IQueryable<User> q = db.Users;
// var r = q.Where(u => u.Age > 18).Select(u => u.Name).ToList();
```

在日常代码里你写的 LINQ 大多跑在 `IEnumerable<T>` 上，它的含义是“对内存中的序列做枚举与计算”。当你看到 `IQueryable<T>`（常见于 EF Core、ORM、搜索 SDK），它的含义变成“把你的查询表达式翻译成另一种语言/协议再执行”（比如 SQL）。因此同样是 `Where/Select`，`IEnumerable` 走的是委托（真正执行在本地），`IQueryable` 走的是表达式树（真正执行在外部系统）。

项目里常见点	IEnumerable	IQueryable
计算发生在哪里	本机、内存	外部系统（数据库/服务）
`Where` 接收什么	`Func<T,bool>`	`Expression<Func<T,bool>>`
什么时候执行	枚举时（延迟）	枚举时（延迟，但会生成查询）
常见坑	多次枚举重复计算	在本地方法里写复杂逻辑导致“无法翻译”

```
using System;
using System.Linq;
```

```
using System.Linq.Expressions;

namespace StudentManagerApp
{
    public sealed class Student
    {
        public string Id { get; init; } = "";
        public string Name { get; init; } = "";
        public int Age { get; init; }
    }

    public static class QueryNotes
    {
        // 这一类签名一般表示“可被翻译”的查询（IQueryable 常见）
        public static Expression<Func<Student, bool>> AdultExpr() => s => s.Age >= 18;

        // 这一类签名一般表示“直接在内存执行”的过滤（IEnumerable 常见）
        public static Func<Student, bool> AdultFunc() => s => s.Age >= 18;
    }
}
```

并发与同步原语

面试问并发通常不是问你“会不会 Thread”，而是问你：共享数据怎么保护、异步场景怎么限流、并发集合怎么用。`lock` 是最常见的互斥入口（语法糖包住 `Monitor.Enter/Exit`），`SemaphoreSlim` 更像“允许 N 个并发进入”的闸门（限流、并发下载/并发写入），`ConcurrentDictionary` 则是把常见的字典并发读写坑封装掉。

场景	推荐工具	直觉
多线程/多任务修改同一份 List/Dictionary	`lock`（或改用并发容器）	同一时刻只能一个人改
限制并发数量（比如同时写文件最多 2 个）	`SemaphoreSlim`	放行 N 张票
高并发读写字典	`ConcurrentDictionary`	内部做了锁分段/无锁优化
异步里保护共享状态	`lock` 只能保护同步代码；异步限流用 `SemaphoreSlim.WaitAsync`	不要在 async 里手写自旋

```
using System;
using System.Collections.Concurrent;
using System.Threading;
using System.Threading.Tasks;

namespace StudentManagerApp
{
    public sealed class StudentIndex
    {
        private readonly ConcurrentDictionary<string, int> _hits =
new(StringComparer.OrdinalIgnoreCase);
```

```

        // 并发安全计数：AddOrUpdate 是常用面试点
        public void Hit(string id) => _hits.AddOrUpdate(id, addValue: 1, updateValueFactory: (_, old) => old + 1);

        public int GetHits(string id) => _hits.TryGetValue(id, out var n) ? n : 0;
    }

    public sealed class StudentExporter
    {
        private readonly SemaphoreSlim _gate = new(initialCount: 2); // 同时最多 2 个导出任务

        public async Task ExportAsync(string id, CancellationToken ct)
        {
            await _gate.WaitAsync(ct);
            try
            {
                await Task.Delay(200, ct); // 模拟 IO
                Console.WriteLine($"Export done: {id}");
            }
            finally
            {
                _gate.Release();
            }
        }
    }
}
```

TaskCompletionSource、ValueTask 与“把回调改成 await”

当你接触到回调式 API（事件、第三方 SDK）时，经常需要把“某个事件发生”转换成 `await`。`TaskCompletionSource<T>` 就是这个桥梁：你手动把结果 `SetResult/SetException/SetCanceled` 填进去，外面就能 `await tcs.Task`。`ValueTask` 则是性能优化点：当一个 async API 很多时候“同步就能返回结果”时，用 `ValueTask` 可以减少分配；但它的使用规则更严格，面试里通常点到为止即可。

你想要的效果	常用工具
把事件/回调变成 await	`TaskCompletionSource<T>`
表达“可能同步完成”的异步返回	`ValueTask<T>`（谨慎）
正确取消/超时	`CancellationToken` + `Task.WhenAny`

```
using System;
using System.Threading;
using System.Threading.Tasks;

namespace StudentManagerApp
{
    public sealed class StudentManager
    {
```

```
        public event EventHandler<string>? StudentAdded;

        public void Add(string id) => StudentAdded?.Invoke(this, id);

        public Task<string> WaitNextAddAsync(CancellationToken ct)
        {
            var tcs = new TaskCompletionSource<string>(TaskCreationOptions.RunContinuationsAsynchronously);

            void Handler(object? s, string id) => tcs.TrySetResult(id);

            StudentAdded += Handler;

            ct.Register(() => tcs.TrySetCanceled(ct));

            return tcs.Task.ContinueWith(t =>
            {
                StudentAdded -= Handler;
                return t.GetAwaiter().GetResult();
            }, CancellationToken.None, TaskContinuationOptions.ExecuteSynchronously, TaskScheduler.Default);
        }
    }

    internal static class Demo
    {
        static async Task Main()
        {
            var mgr = new StudentManager();
            using var cts = new CancellationTokenSource(2000);

            var waiting = mgr.WaitNextAddAsync(cts.Token);

            mgr.Add("S001");
            Console.WriteLine("Next added = " + await waiting);
        }
    }
}
```

Tuple、解构与 Deconstruct

当一个方法天然要返回多个值时，不必为了“凑返回类型”去创建一个只用一次的小类，`(T1, T2)` 的 tuple 往往更合适。解构让你能直接写 `var (ok, msg) = ...` ，读起来像“把返回值拆开”。

写法	用途
`(bool ok, string msg)`	返回多个值并自带字段名
`var (ok, msg) = ...`	解构赋值
`Deconstruct(out ...)`	让自定义类型也支持解构

```
using System;

namespace StudentManagerApp
{
    public sealed class StudentService
    {
        public (bool Ok, string Message) TryRegister(string id, string name)
        {
            if (string.IsNullOrWhiteSpace(id)) return (false, "Id 不能为空");
            if (string.IsNullOrWhiteSpace(name)) return (false, "Name 不能为空");
            return (true, "OK");
        }
    }

    internal static class Demo
    {
        static void Main()
        {
            var svc = new StudentService();
            var (ok, msg) = svc.TryRegister("S001", "Alice");
            Console.WriteLine($"ok={ok}, msg={msg}");
        }
    }
}

### 运行时与资源释放补强（GC、枚举、反射与 Dispose）

#### 补充：GC/分配/Span 与 IAsyncDisposable

国内面试问性能，往往不是让你背 GC 世代，而是想看你有没有“定位–验证–复盘”的思路。你可以先承认现实：托管语言的性能问题很多来自分配与 GC。大对象（比如大数组、巨字符串）可能进入 LOH，回收策略不同，容易带来卡顿；频繁的小对象分配会增加 Gen0/Gen1 压力。你不需要把每个细节讲完，但要能说清楚：减少不必要分配、避免在热路径制造垃圾，必要时用结构化手段（缓存、对象池、Span/Memory）降低分配。

`Span<T>`/`Memory<T>` 是近几年很常见的“高性能工具”，面试里一句话就够：它们让你在不分配新数组的情况下对连续内存做切片处理，适合解析协议、字符串处理等热点路径。与此同时也要强调：它提升性能的前提是你能正确管理生命周期，别把 Span 逃逸到不该去的地方。

另外一个高频点是 `IDisposable` 与资源释放：你已经写了 `using`，再补一句 `IAsyncDisposable`/`await using`，就能覆盖现代场景（异步流、异步释放连接等）。

```

await using var stream = File.OpenRead("data.bin"); // IAsyncDisposable 场景

```

#### GC、托管内存与“资源释放”的边界

C# 里“内存”是 GC 管的，所以你很少需要、也不应该去“手动 free”。但很多面试与真实 bug 都来自一个误会：GC 只负责回收托管对象占用的内存，并不等价于“把所有外部资源都关掉”。文件句柄、网络连接、数据库连接、计时器、非托管句柄这些东西，往往需要你显式释放；而且这些资源的“及时释放”与“对象何时被 GC 回收”是两件完全不同的事。

```
```
```

```
你可以把它记成一条稳定的边界：只要一个类型持有“外部资源”或者持有实现了 `IDisposable` 的对象并且它的生命周期需要被你控制，就应该用 `using` 或显式 `Dispose()` 来结束它的生命周期。这里不需要背 GC 的代际细节，但要形成直觉：GC 回收是不可预测的，`Dispose` 是可预测的；`Dispose` 解决的不是“省内存”，更多时候是“及时归还稀缺资源”。

| 现象/问题 | 常见根因 | 更稳的写法（StudentManager 语境） |
| --- | --- | --- |
| 写文件后发现句柄被占用、文件删不掉 | Stream 没有及时释放 | `using var logger = new StudentLogWriter(path);` 写完立即释放 |
| 程序运行久了句柄数升高、连接耗尽 | Socket/DbConnection/Timer 没有 Dispose | 把资源的生命周期和业务操作绑定到 `using` 作用域 |
| “我写了 finalizer 为啥还会慢？” | Finalizer 会让对象进入终结队列，回收更慢 | 只有真正持有非托管资源才写 finalizer；托管资源用 `Dispose` 即可 |
| 多次 Dispose 导致异常 | Dispose 不幂等 | 在 Dispose 里用 `_disposed` 做保护，保证重复调用安全 |

```

using System;
using System.IO;

namespace StudentManagerApp
{
    public sealed class StudentLogWriter : IDisposable
    {
        private StreamWriter? _writer;
        private bool _disposed;

```
```
    public StudentLogWriter(string path)
    {
        _writer = new StreamWriter(File.Open(path, FileMode.Append, FileAccess.Write, FileShare.Read));
    }

    public void Write(string line)
    {
        if (_disposed) throw new ObjectDisposedException(nameof(StudentLogWriter));
        _writer!.WriteLine(line);
        _writer.Flush();
    }

    public void Dispose()
    {
        if (_disposed) return;
        _writer?.Dispose();
        _writer = null;
        _disposed = true;
        GC.SuppressFinalize(this);
    }
}

internal static class Demo_GC_Dispose
{
    public static void Main()
```
```

```
    {
        var path = "students.log";

        using (var w = new StudentLogWriter(path))
        {
            w.Write("StudentManager started");
        }

        Console.WriteLine("log written and disposed.");
    }
}
```

}

```

#### 不安全代码

`unsafe` 允许你使用指针与直接内存操作，这会绕过一部分运行时安全保护，因此风险更高，出错更可能直接崩溃或产生不可预期行为。工程上它通常只在与原生库互操作或极限性能场景中出现；对初学者更重要的是：读到它不慌，知道 `fixed` 的作用是把托管对象地址“钉住”，避免 GC 移动导致指针失效，同时明白指针不要带出 `fixed` 作用域。

| 语法/关键点 | 含义 | 你需要记住的注意点 |
| ———————— | ———————— | ———————————— |
| `type* p` | 声明指针 | 指针无效会崩溃 |
| `&x` | 取地址 | 只能对可取地址的变量 |
| `*p` | 解引用 | 不做边界检查 |
| `fixed` | 固定托管对象地址 | 指针不要带出 fixed 块 |
| `IntPtr` / `nint` | 指针尺寸相关类型 | 64 位下不要随意转 int |

```csharp
#nullable enable
using System;

namespace StudentManagerApp
{
    public static class StudentBinaryHelper
    {
        public static unsafe int ReadVersion(byte[] data)
        {
            if (data == null || data.Length < 4) throw new ArgumentException("data 长度不足4");

            fixed (byte* p = data)
            {
                int* pi = (int*)p;
                return *pi;
            }
        }
    }
}
```
```

IEnumerable 的延迟执行与“重复枚举”陷阱

你在 LINQ 里看到的大多数方法（`Where/Select/GroupBy/OrderBy` 等）返回的往往是 `IEnumerable<T>`，这意味着它只是描述了“怎么枚举”，而不是把结果立刻算出来。这个特性本身没问题，问题出在你把它当成“已经算完的列表”来用，于是就会出现：同一条查询在不同地方 `foreach` 两次，背后其实把过滤逻辑跑了两次；更糟的是，如果数据源是“会变化/会有副作用”的（例如从文件读取、从网络拉取、随机数、迭代器内部写日志），你会得到难以复现的行为。

这类坑的解决方式很朴素：当你需要“稳定快照”时就物化（`ToList/ToArray`）；当你只需要“一次性流水线”时就保持 `IEnumerable`。所以这不是“永远 ToList”，而是把 ToList 放在你真正要切断延迟执行的边界上。

你看到的写法	表面上像什么	实际上发生了什么	更稳的选择
`var q = list.Where(...);`	得到了一组结果	得到的是“可枚举的查询”	需要快照就 `var snapshot = q.ToList();`
`q.Count()` 再 `foreach(q)`	先算数量再遍历	可能把查询跑两遍	`var snapshot = q.ToList();` `snapshot.Count; foreach(snapshot)`
`FirstOrDefault()`	取第一个或默认	会触发一次枚举	若后面还要用同一批结果，先物化

```
using System;
using System.Collections.Generic;
using System.Linq;

namespace StudentManagerApp
{
    public class Student
    {
        public int Id { get; init; }
        public string Name { get; init; } = "";
        public int Age { get; init; }
        public override string ToString() => $"#{Id} {Name} ({Age})";
    }

    internal static class Demo_IEnumerable_Deferred
    {
        public static void Main()
        {
            var students = new List<Student>
            {
                new() { Id = 1, Name = "Alice", Age = 20 },
                new() { Id = 2, Name = "Bob", Age = 17 },
                new() { Id = 3, Name = "Cathy", Age = 22 },
            };

            var query = students.Where(s =>
```

```
            {
                Console.WriteLine($"filter hit: {s.Name}");
                return s.Age ≥ 18;
            });

            Console.WriteLine("--- Count() ---");
            Console.WriteLine(query.Count());

            Console.WriteLine("--- foreach ---");
            foreach (var s in query) Console.WriteLine(s);

            Console.WriteLine("--- materialize once ---");
            var snapshot = query.ToList();
            Console.WriteLine(snapshot.Count);
            foreach (var s in snapshot) Console.WriteLine(s);
        }
    }
}
```

反射补强

你前面已经能用反射把 Attribute 读出来并做映射了，这已经够“能用”。要到面试里的熟练级，通常还会追问两类点：第一类是你能不能控制扫描范围（例如只扫 public instance 属性，不要扫静态、不要扫继承层级里的某些成员），这就是 `BindingFlags` 的意义；第二类是你知不知道反射是有成本的，热点路径里通常要把“反射得到的 PropertyInfo/Attribute 信息”缓存起来，避免每次映射都重新扫一遍。

你想拿到什么	常用 BindingFlags	直觉解释
公共实例成员（最常见）	`Public | Instance`	只看对外暴露的实例 API
包含非 public（例如 private 字段/属性）	`NonPublic | Instance`	用于框架/序列化/测试工具
静态成员	`Static`	扫工具类/常量
只取当前类型声明的成员	`DeclaredOnly`	避免把基类成员一起扫进来

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Linq;
using System.Reflection;

namespace StudentManagerApp
{
    [AttributeUsage(AttributeTargets.Property, AllowMultiple = false, Inherited = true)]
    public sealed class StudentColumnAttribute : Attribute
    {
        public string ColumnName { get; }
        public bool Required { get; set; }
        public StudentColumnAttribute(string columnName) => ColumnName = columnName;
    }

    public class Student
```

```
    {
        [StudentColumn("id", Required = true)]
        public string Id { get; set; } = "";

        [StudentColumn("name", Required = true)]
        public string Name { get; set; } = "";

        [StudentColumn("age")]
        public int Age { get; set; }
    }

    public static class StudentReflectionCache
    {
        private sealed record ColumnMap(string Key, PropertyInfo Prop, StudentColumnAttribute Attr);

        private static readonly ConcurrentDictionary<Type, ColumnMap[]> _cache = new();

        public static ColumnMap[] GetMaps(Type t)
        {
            return _cache.GetOrAdd(t, static type =>
            {
                return type
                    .GetProperties(BindingFlags.Public | BindingFlags.Instance)
                    .Select(p => new { Prop = p, Attr = p.GetCustomAttribute<StudentColumnAttribute>() })
                    .Where(x => x.Attr is not null)
                    .Select(x => new ColumnMap(x.Attr!.ColumnName, x.Prop, x.Attr!))
                    .ToArray();
            });
        }

        public static Student MapFrom(Dictionary<string, string> row)
        {
            var s = new Student();
            foreach (var m in GetMaps(typeof(Student)))
            {
                if (!row.TryGetValue(m.Key, out var raw))
                {
                    if (m.Attr.Required)
                        throw new ArgumentException($"missing required: {m.Key}");
                    continue;
                }

                object v = m.Prop.PropertyType == typeof(int) ? int.Parse(raw) : raw;
                m.Prop.SetValue(s, v);
            }
            return s;
        }
    }

    internal static class Demo_Reflection_Cache
    {
```

```
        public static void Main()
        {
            var row = new Dictionary<string, string>
            {
                ["id"] = "S001",
                ["name"] = "Alice",
                ["age"] = "20"
            };

            var s = StudentReflectionCache.MapFrom(row);
            Console.WriteLine($"mapped ⇒ {s.Id} {s.Name} {s.Age}");
        }
    }
}
```

析构函数（Finalizer）与 IDisposable

资源释放这块最怕的误区是把析构函数当成“离开作用域自动释放”的入口。GC 的工作对象是“内存”，但文件句柄/Socket/数据库连接这类资源需要的是“及时归还”，而 GC 回收时间是不确定的，所以析构函数（`~Type()`）不适合承担“及时释放”的职责。工程里的正确姿势是：需要及时释放的资源实现 `IDisposable`，调用方用 `using` 保证离开作用域就调用 `Dispose()`；析构函数如果存在，通常只作为兜底，并且在 `Dispose()` 成功后会 `GC.SuppressFinalize(this)` 让对象不再走终结路径。下面这张表把你读源码时最常见的决策点列出来。

你手里持有的东西	推荐方式	你真正想解决的问题	备注
只占托管内存（普通对象图）	不需要手动释放	GC 负责回收内存	不写 finalizer，不实现 IDisposable（除非内部持有可释放资源）
文件/流/连接/计时器等需要及时释放的对象	`IDisposable` + `using`	确定性释放：用完立刻释放	绝大多数业务代码就停在这里
直接持有非托管句柄/native memory	完整 Dispose 模式 +（少数）finalizer/或 SafeHandle	忘记 Dispose 时也要兜底	更推荐 SafeHandle，把复杂性下放给框架

示例（示例类型 + Main 操作演示，展示 using 的效果）：

```
#nullable enable
using System;
using System.IO;

namespace StudentManagerApp
{
    public sealed class StudentLogger : IDisposable
    {
        private readonly StreamWriter _writer;
        private bool _disposed;

        public StudentLogger(string path)
        {
```

```
            _writer = new StreamWriter(new FileStream(path, FileMode.Append, FileAccess.Write, FileShare.Read));
        }

        public void Log(string message)
        {
            if (_disposed) throw new ObjectDisposedException(nameof(StudentLogger));
            _writer.WriteLine($"[{DateTime.Now:O}] {message}");
            _writer.Flush();
        }

        public void Dispose()
        {
            if (_disposed) return;
            _writer.Dispose();
            _disposed = true;
            GC.SuppressFinalize(this); // 没有 finalizer 时不是必须，但写了也不影响
        }
    }

    public static class Program
    {
        public static void Main()
        {
            using var logger = new StudentLogger("students.log");
            logger.Log("StudentManager started");
            logger.Log("Add student: Alice");
        }
    }
}
```